

微构体中填充导电介质的仿真优化

摘要

本文研究边长 10000 nm 的微构体在填充直圆柱介质 A 与球形介质 B 后的导通问题。四问共用一个判定内核：按题给的边界截断规则把越界介质折回微构体得到若干碎片，两两计算表面最短距离，不超过 1.8 nm 即连边，再用并查集判断两个带电面是否落入同一连通分量。介质 A 建模为球柱体，使表面距离精确等于轴线段最短距离减 $2r$ ，判定得以完全向量化。内核经 6 项独立校验，其中用附件数据做的“还原—重截断”往返测试对 334 根完整介质逐坐标一致。

建模前的数据审计改变了模型。附件的一行不是一根介质 A，而是截断后的碎片：按轴向归组后组 1/2/3 的母介质数为 7/28/354；组 1、组 2 的周期盒是 $10000 \times 1000 \times 1000$ nm 而非立方体；介质方向也不是各向同性，而是以带电面法向为极轴、 $\theta \sim U(0, \pi)$ 的分布（KS 检验：各向同性 $p = 6.7 \times 10^{-19}$ 被拒绝，该分布 $p = 0.73$ 不被拒绝）。方向偏向使同一体积分数的导通概率接近翻倍，本文以附件标定分布为主口径，全程并列给出球面均匀的对照。

针对问题一，按各组实际几何用判定内核直接求解，得组 1 **不导通**、组 2 **导通**、组 3 **导通**。组 1 的体积分数 (0.99%) 反而高于组 3 (0.50%)，却因接触边仅 2 条而在中段断路。补回附件丢弃的短碎片、把判据阈值在 0.9–2.7 nm 间扰动，三组结论均不变。

针对问题二，用蒙特卡洛频率估计导通概率、Wilson 区间给区间估计，每档 8000 次试验，得 $\varphi = 0.50\%$ 时 $P = 0.2193$ ， 0.60% 时 0.4798， 0.70% 时 0.7458， 1.00% 时 0.9994；排除体积判据的独立估计与仿真跃变中点一致到 9%。

针对问题三，先用 logit 拟合粗定位跃变点，再在 0.01% 网格上逐点复算，取最小的达标点，得**最低体积分数** $\varphi = 0.78\%$ (552 根)；12000 次独立复核得 $P = 0.9092$ (下界 0.9039)，相邻的 0.77% 仅 0.8898。

针对问题四，导通概率对 N_A 、 N_B 都单调不减，最优解必落在可行边界上，二维整数搜索降为沿边界的一维搜索；用 logit 代理模型缩小范围，候选点一律大样本直接验证。结果为 $N_A = 552$ 、 $N_B = 0$ ，**总成本 8.19 元**——给定价格下掺入介质 B 并不划算。机理是边际效率：介质稀疏时 B 每元收益高于 A，但 $P \geq 90\%$ 的约束把可行解推到 $N_A \approx 400$ 交叉点右侧的 A 占优区段。文中给出 B 的盈亏平衡单价。

关键词：连续渗流，蒙特卡洛仿真，周期边界截断，球柱体最短距离，仿真优化

一、问题重述与问题分析

1.1 问题背景与研究对象

导电体系的宏观导电性由填充其中的导电介质能否搭出一条贯穿电极的通路决定。题目把这一体系理想化为边长 10000 nm 的立方体“微构体”：内部充满绝缘溶剂与若干导电介质，选取垂直于 X 轴的两个面 ($x = \pm 5000$ nm) 为带电面，其余四面绝缘。两个导体之间、或导体与带电面之间，当表面最短距离不超过 1.8 nm 时视为导通；若两带电面之间存在一条由导电介质接力构成的通路，则判定微构体导通。

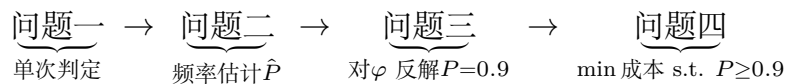
导电介质有两种：介质 A 为高 5000 nm、底面半径 30 nm 的直圆柱；介质 B 为半径 200 nm 的正球体。介质位置与方向随机，互不碰撞，允许相互贯穿；越出微构体边界的部分按**边界截断规则**（沿越界方向的反方向平移一个边长，必要时重复）折回微构体内。

1.2 需要回答的问题

1. 附件给出三个微构体中介质 A 的坐标，分别判定是否导通；
2. 只填介质 A，体积分数为 0.50%、0.60%、0.70%、1.00% 时的导通概率；
3. 只填介质 A，使导通概率不低于 90% 的最低体积分数（精确到 0.01%）；
4. A、B 混填，在导通概率不低于 90% 的前提下使填充总成本最低的填充量。成本为 A 1.05 元/ μm^3 、B 0.05 元/ μm^3 。

1.3 总体思路

四问共用同一个判定内核：**给定一组介质的位姿，判断微构体是否导通**。内核由三步组成——边界截断把每根介质化为若干位于盒内的碎片；两两计算表面最短距离，不超过 1.8 nm 则连边；并查集求连通分量，判断是否存在同时连到两个带电面的分量。四个问题只是内核的不同用法：



从数学结构看，问题二、三是**连续渗流**（continuum percolation）中贯穿概率的估计与反解——这正是碳纳米管、炭黑等导电填料复合材料里“填到多少才导电”这一问题的数学形式 [5]；问题四是以蒙特卡洛估计量为约束的**整数规划**，即题目所说的“仿真优化”。

整体求解路径如图 1（数学建模的一般流程与常用算法可参见 [1][2]）：先由附件数据审计定下建模口径（第 2 节），据此实现并校验判定内核（第 3 节），四个问题再分别以单次判定、频率估计、阈值反解和带约束优化的方式调用同一内核；每一环都

配有独立校验，结论不依赖任何未经检查的中间结果。

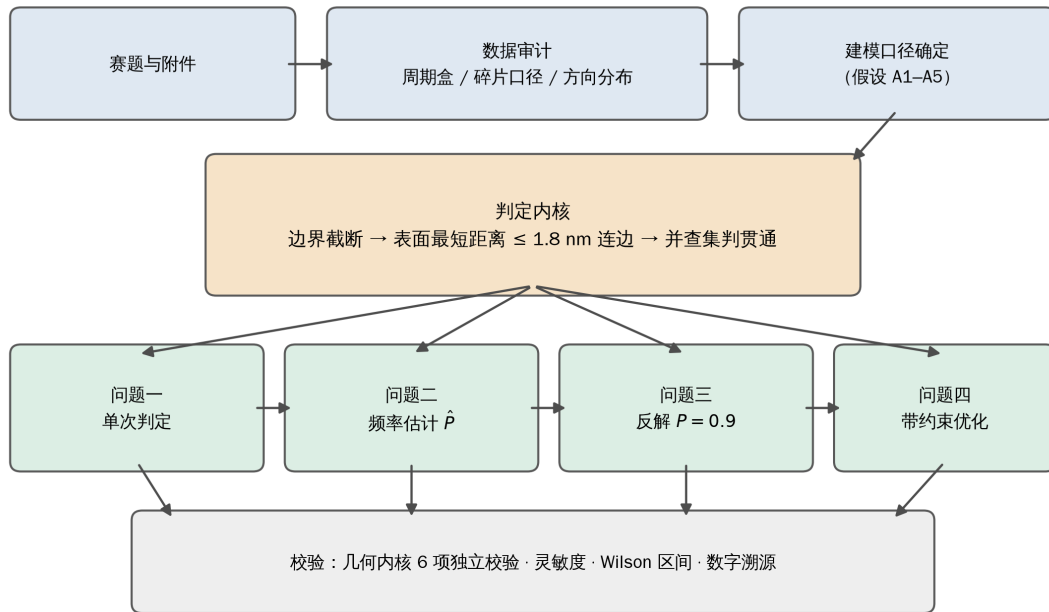


图 1 技术路线：四问共用同一个导通判定内核

1.4 问题一的分析

问题一表面上是”代入判定”，但附件的组织方式与题面并不直接对应：附件的一行并不是一根介质 A。因此问题一实际上先是一个数据还原问题，其结论同时决定了问题二至四随机生成器的口径。本文把这一步单独作为第 2 节。

1.5 问题二、三的分析

导通与否是随机配置的函数，导通概率没有解析表达式，只能用蒙特卡洛频率估计，并给出置信区间。问题三要求精确到 0.01%，对应介质 A 根数相差约 7 根，需要先用 logit 拟合定位跃变点，再在 0.01% 网格上直接复算，避免把结论压在拟合模型的形式上。

1.6 问题四的分析

介质 B 单颗成本只有介质 A 的 1/8.9，但半径 200 nm 的球跨接能力远不如长 5000 nm 的棒；球的作用是充当棒与棒之间的”桥”。由于导通概率对 N_A 、 N_B 都单调不减，最优解必落在可行域边界上，二维搜索可降为沿边界的一维搜索。

二、附件数据审计与建模口径的确定

先看数据再建模。附件里有四件事与题面的字面读法不同，每一件都改变了后面的模型。本节的全部结论由 `src/audit_attachment.py` 产生，可复现。

2.1 附件的一行是碎片，不是一根介质

题面规定介质 A 的轴长恒为 5000 nm，但附件中绝大多数行的两端点距离都小于 5000 nm（组 3 中仅 155/535 行等于 5000）。把方向向量完全相同的行归为一组后，组数为 7 / 28 / 354，且每组轴长之和都不超过 5000 nm。故：**一行是边界截断后的一个碎片，同向的若干行合起来才是一根母介质**。组 3 的 354 根对应体积分数 0.5005%，正是问题二的第一档 0.50%，这既验证了还原，也说明组 3 就是按问题二的口径生成的。

2.2 组 1、组 2 的周期盒不是立方体

碎片在边界处成对出现（前一碎片终点在 $+h$ 、后一碎片起点在 $-h$ ，其余两坐标完全相同），因此只要统计碎片端点**精确落在**某个平面上的次数，就能直接读出回绕面的位置。坐标是浮点数，随机落到某个整值平面上的概率为零，所以这种精确命中只能来自截断本身：

表 1 碎片端点落在候选周期面上的次数（判据： $||x_j| - h_j| < 10^{-6}$ nm）

微构体	X 面 ± 5000	Y 面 ± 500	Z 面 ± 500	Y 面 ± 5000	Z 面 ± 5000
组 1	7	2	4	0	0
组 2	22	13	11	0	0
组 3	181	—	—	118	111

结论很直接：组 3 的三个方向都在 ± 5000 回绕，与题面的 10000^3 立方体一致；而组 1、组 2 的 x 在 ± 5000 回绕、 y 与 z 却在 ± 500 回绕，截面只有 1000×1000 nm。两组的体积分数因此是 0.99% 与 3.96%，而非按立方体折算的 0.0099% 与 0.0396%——差了整整 100 倍。问题一必须按各自的实际几何判定。

2.3 附件丢弃了短碎片

把每根母介质的碎片首尾相接，长度应恰为 5000 nm。组 3 有 49 根接不满：46 根缺口小于 500 nm，另 3 根缺口为 636/732/870 nm（各缺两个碎片）。而附件中保留的最短碎片是 526 nm。故所有被丢弃的碎片单个长度均小于 500 nm。图 2 把保留碎片与反推出的缺失碎片画在同一坐标上，两者在 500 nm 处几乎不重叠，这正是“按长度阈值筛除”留下的痕迹。被丢弃的碎片有可能正好是一座桥，因此问题一另按“补回缺失碎片”的口径复判一次，结论不变（第 4.3 节）。

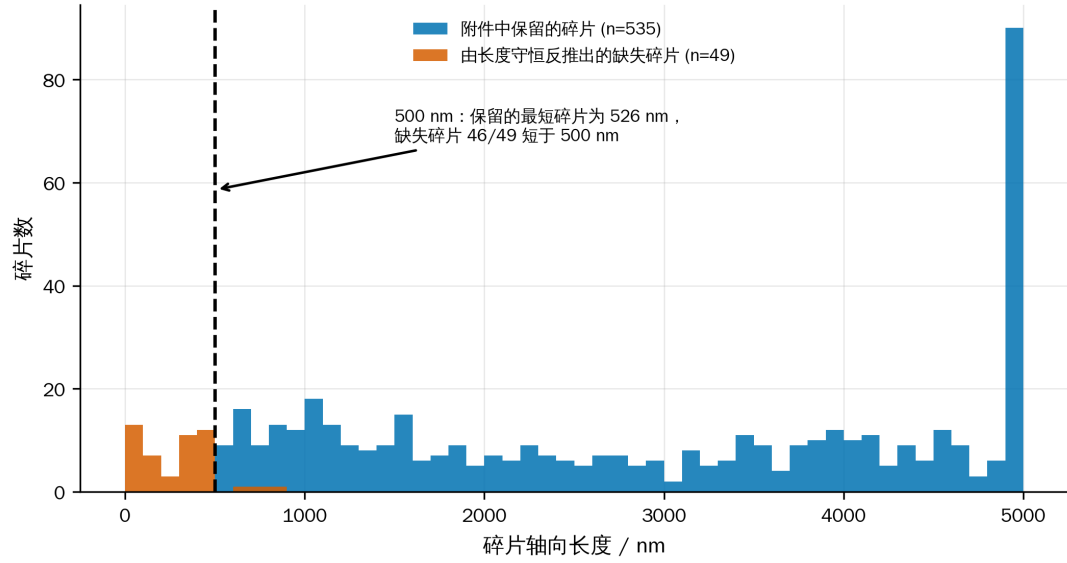


图 2 附件保留碎片与反推出的缺失碎片的长度分布

2.4 介质方向不是各向同性——影响最大的一条

组 3 中 354 根母介质方向余弦绝对值的均值为 $E|u| = (0.656, 0.390, 0.394)$ 。各向同性应为 $(0.5, 0.5, 0.5)$ ；而以 X 轴（带电面法向）为极轴、 $\theta \sim U(0, \pi)$ 、 $\phi \sim U(0, 2\pi)$ 的分布，理论值为 $(2/\pi, (2/\pi)^2, (2/\pi)^2) = (0.637, 0.405, 0.405)$ 。对 $|u_x|$ 作 Kolmogorov–Smirnov 检验：

假设分布	KS 统计量 D	p 值	结论
球面均匀（各向同性）	0.243	6.7×10^{-19}	拒绝
$\theta \sim U(0, \pi)$ ，极轴为 X	0.036	0.73	不拒绝

各向同性被断然拒绝。图 3 左图把经验累积分布与两条理论曲线画在一起：经验分布几乎贴合极角均匀的曲线，而与各向同性的对角线明显分离；右图用三个方向余弦的均值给出同一结论。这一偏向使介质更多地沿带电面法向排列，**同一体积分数下的导通概率接近翻倍**（第 5.2 节表 5）。因此本文以**附件标定分布**为主口径，同时全程并列给出球面均匀口径的结果。

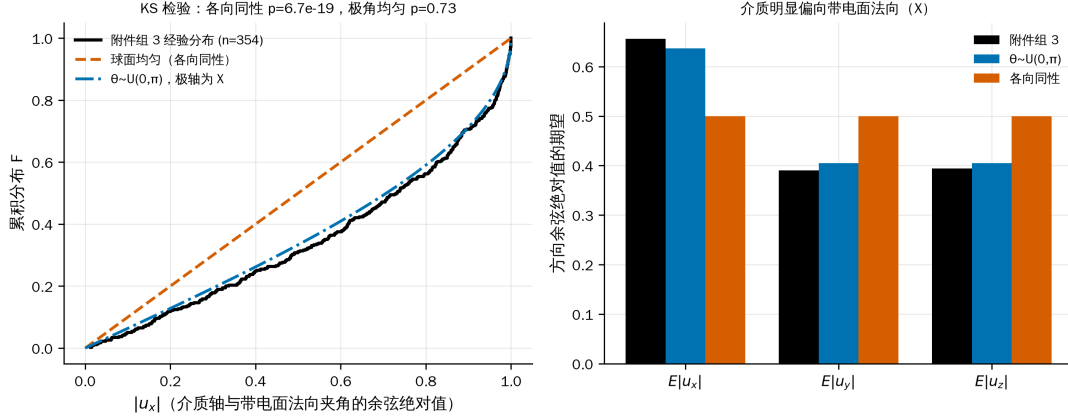


图 3 组 3 介质方向的经验分布与两种候选分布

三、模型假设、符号与判定内核

3.1 模型假设

每条假设后面注明它在哪里被用到；用不到的假设不写。

- **A1 (球柱体近似)** 介质 A 建模为半径 $r = 30 \text{ nm}$ 、轴长 $h = 5000 \text{ nm}$ 的球柱体（圆柱两端各加一个半球帽）。用于 3.3 节把”表面最短距离”化为”轴线段最短距离减 $2r$ ”。与题面的平端面圆柱相比，体积仅相差 $4r/(3h) = 0.80\%$ ，且差别只出现在端帽附近；第 8.1 节量化了它对判定的影响（临界接触中受影响者约 3.3% ）。
- **A2 (碎片独立)** 边界截断产生的每个碎片是独立导体，同一根母介质的不同碎片之间没有电学连接。用于 3.4 节建图。这是题面”截断”的字面含义；若改为碎片粘合，任何一根跨越 X 边界的介质都会同时接触左右带电面而使微构体必然导通，问题二至四将退化为平凡问题（第 8.2 节给出该口径下 $P = 1.000$ 的对照）。
- **A3 (硬壁)** 四个绝缘面不导电，介质之间的距离按普通欧氏距离计算，不跨绝缘面配对。与 A2 一致：截断只是保持体积分数的放置约定，不意味着盒子在电学上是周期的。
- **A4 (均匀各向随机)** 介质中心在微构体内均匀分布；方向服从第 2.4 节标定的分布。用于问题二至四的随机生成。不考虑重力与介质间碰撞（题面给定）。
- **A5 (判据对称)** 导通判据 $\delta = 1.8 \text{ nm}$ 对介质-介质与介质-带电面一视同仁（题面给定）。

3.2 符号说明

符号	含义	单位
L	微构体边长, $L = 10000$	nm
r, h	介质 A 的半径与轴长, $r = 30, h = 5000$	nm
R	介质 B 的半径, $R = 200$	nm
δ	导通判据的最大表面间距, $\delta = 1.8$	nm
N_A, N_B	介质 A 的根数、介质 B 的颗 数	—
φ_A, φ_B	两种介质各自的体积分数	—
V_A, V_B	单个介质体积, $V_A = \pi r^2 h, V_B = \frac{4}{3}\pi R^3$	nm ³
c_A, c_B	单个介质成本	元
S_i	第 i 个碎片的轴线段	—
P	微构体导通概率	—
$\hat{P}, T$	导通频率估计、蒙特卡洛试 验次数	—

3.3 边界截断与最短距离

边界截断. 设介质 A 的轴线段为 $p + t(q - p), t \in [0, 1]$ 。对每个坐标轴 j 求出线段穿越周期格点平面 $x_j = L/2 + kL$ ($k \in \mathbb{Z}$) 的所有参数 t , 以这些 t 为断点把线段切开; 每一子段整体位于同一个周期像内, 把它平移 $-\lfloor \cdot \rfloor L$ 即回到微构体内。该过程与“题面”把超出部分沿越界方向的反方向平移一个边长, 必要时重复”逐字等价, 且自然支持需要多次平移的情形。

最短距离. 在假设 A1 下, 两个介质 A 的表面最短距离为

$$d_{\text{surf}}(i, j) = d_{\text{seg}}(S_i, S_j) - 2r,$$

其中 d_{seg} 是两条线段的最短距离, 用 Ericson 的线段—线段最近点算法 [7] 精确求出。同理, 介质 B 与介质 A、介质 B 与介质 B 之间分别为 $d_{\text{pt-seg}} - (r + R)$ 与 $\|c_i - c_j\| - 2R$ 。介质与带电面 $x = \pm L/2$ 的表面距离由介质在 x 方向的极值减去半径给出。

图 4 把这两条规则画在一起。(a) 说明截断: 越界部分沿越界方向的反方向平移一个边长回到盒内,

主段与折回段在几何上属于同一根母介质, 但按假设 A2 是两个彼此独立的导体。

(b) 说明介质 A 之间的判据。(c) 说明介质 B 的作用——它本身跨度有限，价值在于充当桥：由 $d_{\text{pt-seg}} \leq r + R + \delta$ 可知，一颗 B 最多能把轴距 $2(R + r + \delta) = 463.6 \text{ nm}$ 以内的两根 A 连起来，这个数在第 7.2 节解释“为什么便宜的 B 反而用不上”时还会再出现。

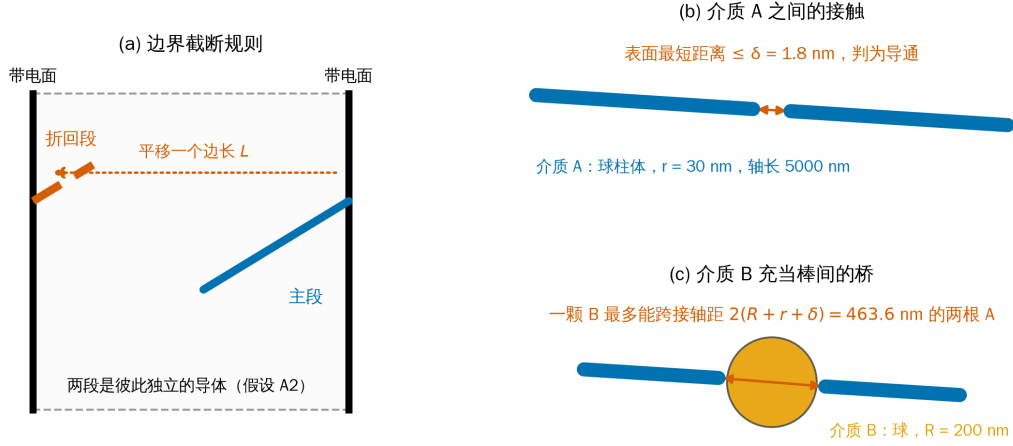


图 4 边界截断规则与导通判据示意

3.4 导通判定

构造无向图 $G = (V, E)$: V 为全部碎片再加两个虚拟节点 L, R (左右带电面)。当两个介质的表面最短距离 $\leq \delta$ 时在其间连边；当某介质与某带电面的表面距离 $\leq \delta$ 时，把它与对应虚拟节点连边。用并查集求连通分量，则

$$\text{微构体导通} \iff \text{find}(L) = \text{find}(R).$$

复杂度. 朴素两两比较为 $O(M^2)$ (M 为碎片数)。实现中对棒—棒用轴对齐包围盒预筛 + 分块计算，对数量可达上万的介质 B 用 KD 树做邻域检索，把 $\varphi = 1\%$ 规模的单次判定从 744 ms 降到 107 ms，且与朴素实现逐位一致 (第 8.1 节 T6)。

四、问题一：三个微构体的导通判定

4.1 判定口径

按第 2 节的审计结论：组 1、组 2 用 $10000 \times 1000 \times 1000 \text{ nm}$ 的实际几何，组 3 用 10000^3 ；附件给出的每个碎片是一个独立导体 (假设 A2)。判定直接使用第 3.4 节的内核，不含任何随机性，结果可逐位复现。

4.2 判定结果

把三组碎片分别送进判定内核，得到表 2。三组的母介质数、周期盒与体积分数差异都很大，因此不能只看“介质多不多”，还要看接触图是否把左右两个带电面连起来。

表 2 问题一的判定结果与接触图规模

微构体	母介质 数	附件碎 片数	周期盒 / nm	体积分 数	接触边 数	触左面	触右面	判定
组 1	7	12	10000×1000	0.99%	2	3	4	不导通
组 2	28	49	10000×1000	3.96%	27	11	11	导通
组 3	354	535	10000×1000	0.50%	166	92	90	导通

三组的判定机理各不相同，值得逐一说明：

组 1 不导通。7 根介质几乎完全沿 X 轴排列 ($E|u_x| = 0.997$)，在 1000×1000 nm 的细通道里本可以首尾接力，但整个微构体只有 2 条接触边，左侧团簇与右侧团簇之间存在断口。换言之，组 1 缺的不是介质总量（体积分数 0.99% 反而高于组 3），而是介质在 X 方向上的接力。

组 2 导通。同样的细通道几何，介质增加到 28 根、体积分数 3.96%，接触边增至 27 条，左右两侧的团簇被打通，存在贯穿通路。

组 3 导通。体积分数只有 0.50%，但微构体截面是组 1、组 2 的 100 倍，92 个碎片触及左带电面、90 个触及右带电面，且每根介质长 5000 nm（半个盒边），只需 2-3 根接力即可跨越，因而在这一体积分数下已经越过渗流阈值。按第 5 节的结果， $\varphi = 0.50\%$ 的导通概率为 0.219，组 3 属于该概率下的正常样本。

接触图中最近的一对介质表面距离为负（组 1 为 -25.4 nm，组 3 为 -59.8 nm），即存在相互贯穿的介质，与题面“允许介质因随机生成而产生相互贯穿、重叠”一致。

把三组的接触图投影到 X - Y 平面即得图 5：按所属连通分量着色后，组 1 左右两侧各自成团、中间留有断口；组 2、组 3 则出现一个同时连到左右带电面的贯通团簇（绿色），与表 2 的判定一致。

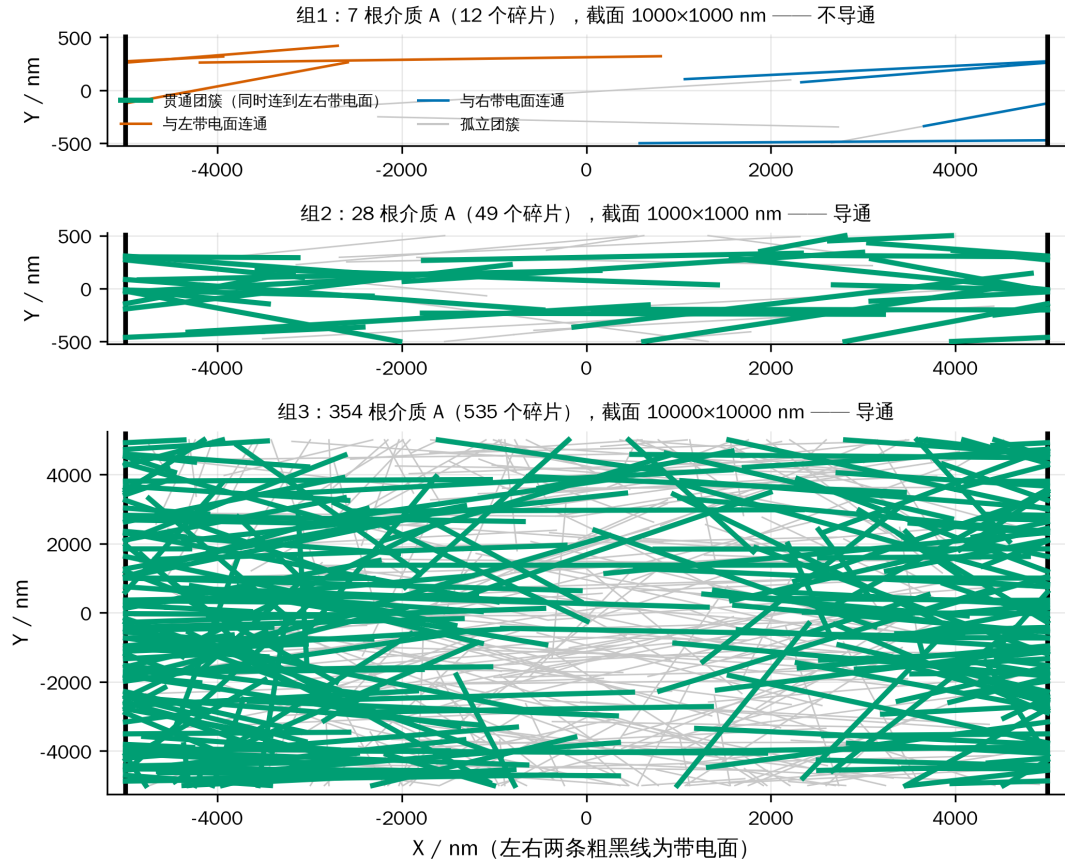


图 5 三个微构体的接触图在 X-Y 平面的投影

4.3 结论的稳健性

三个判定都不是”卡在阈值上”的结论，两项检查支持这一点：

1. 补回被丢弃的碎片。按第 2.3 节把每根母介质的轴线还原为完整的 5000 nm 再重新截断，碎片数由 12/49/535 增至 15/53/589；三组的判定结论均不变。
2. 判据阈值扰动。把 δ 在 $\pm 50\%$ 范围内扫描，三组结论同样不变：

表 3 问题一判定对判据阈值的稳健性

δ / nm	0.90	1.35	1.80	2.25	2.70
组 1	不导通	不导通	不导通	不导通	不导通
组 2	导通	导通	导通	导通	导通
组 3	导通	导通	导通	导通	导通

这说明三组都不处在判定的临界状态：组 1 的断口宽度远大于 δ 的量级，组 2、组 3 的贯通通路也不是靠某一条刚好卡在 1.8 nm 的接触边撑起来的。

需要说明的是，若改用被否决的”碎片粘合”口径（假设 A2 的反面），组 1 也会变成导通——因为组 1 中存在跨越 X 边界的介质，其两个碎片分别贴在左、右带电

面上。三组结论将全部变成”导通”，问题一失去区分度。这从反面支持了假设 A2。

五、问题二：给定体积分数下的导通概率

5.1 模型

给定介质 A 的体积分数 φ ，根数按题目要求四舍五入取

$$N_A = \text{round}(\varphi L^3/V_A), \quad V_A = \pi r^2 h = 1.41372 \times 10^7 \text{ nm}^3.$$

一次试验：按假设 A4 随机生成 N_A 根介质 A，做边界截断，用第 3.4 节的内核判定导通与否。这是蒙特卡洛方法的标准用法——把一个没有解析表达式的概率，转化为对可判定事件的重复抽样 [3]。重复 T 次独立试验，导通频率 $\hat{P} = X/T$ 即为导通概率的无偏估计，其中 $X \sim B(T, P)$ 。区间估计用二项比例的 Wilson 区间 [4] 而非正态近似——当 $\hat{P}$ 接近 0 或 1 时， $\hat{P} \pm 1.96 \text{ se}$ 会给出越界的区间，Wilson 区间不会：

$$\frac{\hat{P} + \frac{z^2}{2T} \pm z\sqrt{\hat{P}(1-\hat{P})/T + z^2/(4T^2)}}{1 + z^2/T}, \quad z = 1.96.$$

5.2 结果

每档做 $T = 8000$ 次独立试验。主口径（附件标定的方向分布）结果如下。

表 4 四档体积分数下的微构体导通概率 ($T = 8000$ ，括号内为 Wilson 95% 区间)

体积分数 φ	介质 A 根数 N_A	导通次数	导通概率 $\hat{P}$	95% 置信区间
0.50%	354	1754	0.2193	(0.2103, 0.2284)
0.60%	424	3838	0.4798	(0.4688, 0.4907)
0.70%	495	5966	0.7458	(0.7361, 0.7552)
1.00%	707	7995	0.9994	(0.9985, 0.9997)

作为对照，若按球面均匀（各向同性）生成方向，同样 8000 次试验给出

表 5 球面均匀口径下的对照结果

体积分数	0.50%	0.60%	0.70%	1.00%
$\hat{P}$ （各向同性）	0.0874	0.2325	0.4911	0.9940
$\hat{P}$ （附件标定，主口径）	0.2193	0.4798	0.7458	0.9994

把两套口径下的全部估计点连成曲线即图 6：带误差棒的标记是题目点名的四档，

误差棒为 Wilson 95% 区间（在 8000 次试验下已窄到与标记同量级）；曲线上其余点来自问题三的粗扫与细网格；两个星号是第 6 节求得的使 $P \geq 90\%$ 的最低体积分数。

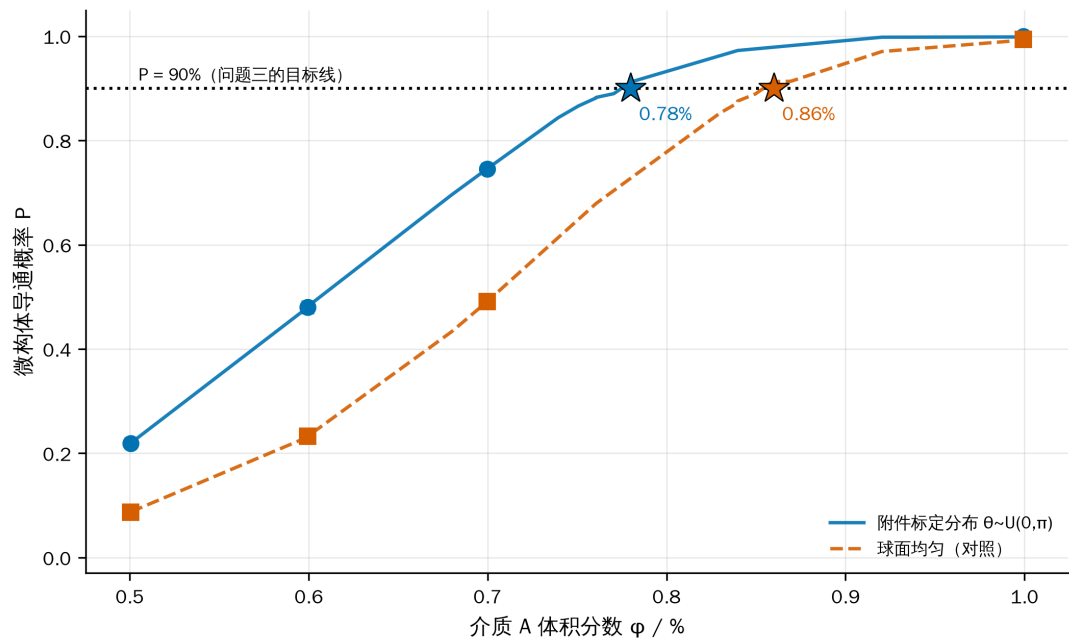


图 6 导通概率随介质 A 体积分数的变化

结果解读.

1. 概率在 0.50%→1.00% 之间由 0.22 升到 0.9994，跨越了一个完整的**渗流跃变**：在陡峭段上，体积分数每增加 0.10%（约 71 根介质），概率就上升约 0.26 (0.219→0.480→0.746)。微构体的导通不是“介质越多导电性越好”的连续过程，而是在一个很窄的窗口里从几乎不通变成几乎必通。
2. **与渗流理论的独立对照**。经典排除体积判据（Balberg 等 [6]）给出随机取向细长杆的阈值满足 $n_c \langle V_{ex} \rangle \approx B_c \approx 1.4$ 。代入本题参数（连通等效直径 $2r + \delta = 61.8$ nm, $\langle V_{ex} \rangle = 2.548 \times 10^9$ nm³）得 $N_c \approx 549$ 根，即 $\varphi_c \approx 0.777\%$ 。三个数排成一条可解释的链：

来源	φ at $P = 0.5$	与上一行的差异来自
排除体积判据（各向同性、无限大体系）	0.777%	—
本文仿真（各向同性、有限盒）	0.705%	有限尺寸效应：介质长 5000 nm 恰为盒边一半，跨越只需 2-3 根接力
本文仿真（附件标定方向、有限盒）	0.607%	方向偏向带电面法向，进一步降低阈值

理论值与各向同性仿真相差 9%，方向一致（有限体系阈值更低），说明仿真内核没有系统性偏差；两步差异都能被独立的物理机制解释，其中”有限尺寸效应”这一步在第 5.3 节有直接的结构证据，而不只是一句合理的猜测。

3. **方向分布的影响远大于统计误差。**表 5 中两套口径在 0.50%、0.60%、0.70% 三档上分别相差 0.132、0.247、0.255，而 8000 次试验的 95% 区间半宽只有约 0.01。也就是说，这一差别不是”算得不够准”，而是**建模口径的选择**，两套答案都列出比只报一套更诚实。
4. 附件组 3 正是 $\varphi = 0.50\%$ 的一个样本，它导通——在主口径下这是一个概率 0.219 的事件，属于正常样本；在各向同性口径下概率只有 0.087，虽不至于不可能，但需要额外解释。这从侧面支持主口径的选择。

5.3 渗流的结构证据：为什么阈值低于无限大体系的估计

导通概率只回答”通没通”，看不出通的时候内部长什么样。渗流理论里刻画这件事的量是**序参量** $S_{\max}$ ——最大连通团簇所含碎片占全部碎片的比例。它的计算完全不涉及带电面，因此是一条独立于导通判定的检查。图 10 把两者画在一起（每档 200 次试验）：

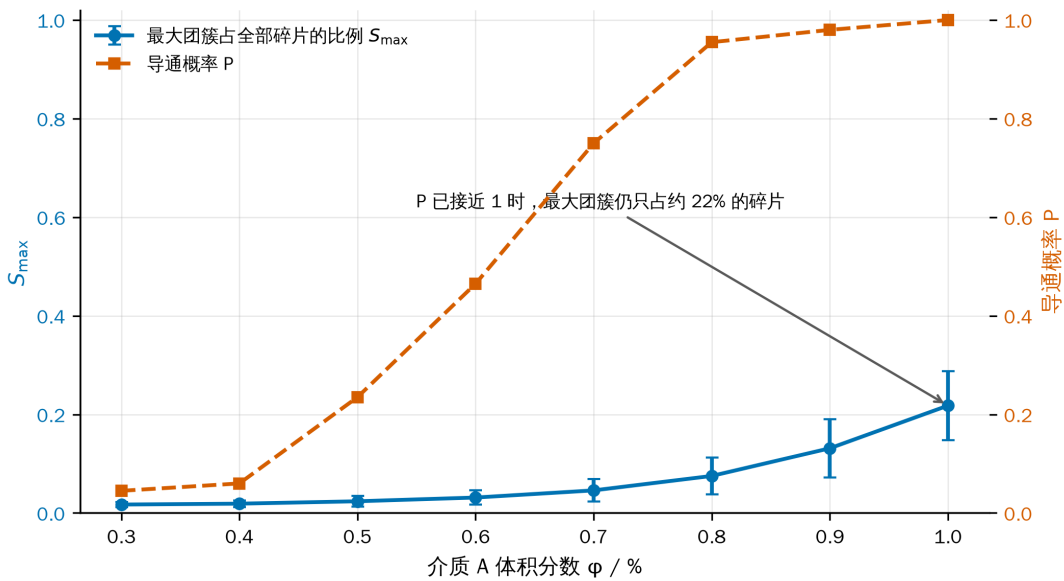


图 10 最大团簇占比与导通概率随体积分数的变化

结果出乎”经典渗流”的直觉，却正好解释了第 5.2 节留下的疑问：当 P 已经接近 1 时，最大团簇仍然只包含约 22% 的碎片（ $\varphi = 1.00\%$ 时 $S_{\max} = 0.218$ ，团簇总数仍有 638 个）；在 P 跃变最陡的 $\varphi = 0.70\%$ 处， $S_{\max}$ 只有 0.046。也就是说，本题的微构体在尚未形成贯穿全系统的巨团簇时就已经导通了。

原因是几何尺度：介质 A 的轴长 5000 nm 恰为盒边的一半，跨越两个带电面只需

要 2-3 根接力。贯通所需的是一条**短链**，而不是一个吞掉大部分介质的巨团簇。这正是有限尺寸效应的直接体现，也解释了为什么第 5.2 节中仿真阈值（各向同性 0.705%）低于排除体积判据对无限大体系的估计（0.777%）——后者刻画的是巨团簇的出现，而本题只需要短链贯通，因此更早发生。

这条结论也提醒：不能把本题的阈值直接外推到更大的微构体。盒边增大而介质尺寸不变时，贯通需要的链长随之增加，阈值会向 0.777% 靠拢。

六、问题三：使导通概率不低于 90% 的最低体积分数

6.1 求解思路

要求”精确到百分号下小数点后两位”，即在 0.01% 的体积分数网格上定位。0.01% 对应 $\Delta N_A = L^3 \times 10^{-4} / V_A \approx 7.07$ 根，相邻网格点的导通概率相差约 0.02，因此蒙特卡洛的 95% 区间半宽必须明显小于 0.02。分两步：

1. **粗定位**. 在 $\varphi \in [0.60\%, 1.00\%]$ 上取 6 个点，各 2000 次试验，用二项似然拟合 $\text{logit } P = a + bN_A$ ，反解 $P = 0.90$ 的穿越点；
2. **网格复算**. 在穿越点附近的 5 个 0.01% 网格点上各做 4000 次试验（区间半宽约 0.009），取**最小的、点估计不低于 0.90 的**网格点作为答案，再用 12000 次试验独立复核。

拟合只用于省算力，最终结论由第 2 步的直接频率给出，不依赖 logit 模型的函数形式。

6.2 结果

粗定位给出 $P = 0.90$ 的穿越点在 $N_A \approx 539$ ($\varphi \approx 0.762\%$)。在其附近的 0.01% 网络上各做 4000 次试验：

表 6 问题三的 0.01% 网格复算（主口径， $T = 4000$ ）

体积分数 φ	N_A	导通概率 $\hat{P}$	95% 置信区间	是否达标
0.74%	523	0.8435	(0.8319, 0.8544)	否
0.75%	531	0.8660	(0.8551, 0.8762)	否
0.76%	538	0.8815	(0.8711, 0.8912)	否
0.77%	545	0.8898	(0.8797, 0.8991)	否
0.78%	552	0.9125	(0.9033, 0.9209)	是

答案：介质 A 的最低填充量为体积分数 $\varphi = 0.78\%$ （对应 552 根介质 A）。

独立复核：在 $\varphi = 0.78\%$ 用另一组随机种子做 12000 次试验，得 $\hat{P} = 0.9092$ ，

95% 置信区间 (0.9039, 0.9142), 区间下界仍高于 0.90, 结论成立。相邻的 0.77% 在两次独立估计中都明显低于 0.90 (0.8898, 区间上界 0.8991), 因此 0.78% 是 0.01% 网格上能达标的**最小**取值, 而不是四舍五入的结果。

按球面均匀口径重做同一流程, 答案为 $\varphi = 0.86\%$ (608 根, 复核 $\hat{P} = 0.9062$)。两套口径相差 0.08 个百分点, 即约 56 根介质, 仍然是全题最大的不确定来源。

按主口径的成本换算, 纯用介质 A 达到 90% 导通概率的代价是 $552 \times c_A = 8.194$ 元, 这是问题四的比较基准。

七、问题四：A、B 混填的最低成本配比

7.1 优化模型

单个介质的成本由体积换算 ($1 \mu\text{m}^3 = 10^9 \text{ nm}^3$):

$$c_A = 1.05 \times \frac{V_A}{10^9} = 1.48441 \times 10^{-2} \text{ 元/根}, \quad c_B = 0.05 \times \frac{V_B}{10^9} = 1.67552 \times 10^{-3} \text{ 元/颗}.$$

优化问题为

$$\min_{N_A, N_B \in \mathbb{Z}_{\geq 0}} C = c_A N_A + c_B N_B \quad \text{s.t.} \quad P(N_A, N_B) \geq 0.90.$$

结构性质. 向已有配置中再加入一个介质, 只会给接触图增加一个顶点和若干条边, 不会删去任何已存在的通路, 故 P 对 N_A 、 N_B 都单调不减。于是可行域是”右”上封闭”的, 最优解必落在可行边界 $N_B = g(N_A)$ 上, 二维搜索降为沿边界的一维搜索。

求解策略. P 没有解析式, 只能用仿真估计, 直接对每个 N_A 做二分需要上百万次仿真。故采用两阶段:

- **阶段 A (代理模型)** . 在 $N_A \times N_B$ 的 7×6 网格上各做 700 次试验, 用二项似然拟合 $\text{logit } P = w_0 + w_1 N_A + w_2 N_B + w_3 \sqrt{N_B} + w_4 N_A N_B / 1000$ 。其中 $\sqrt{N_B}$ 项刻画球的边际收益递减, 交互项刻画”球只有在有棒可搭时才有用”的协同效应。
- **阶段 B (沿边界搜索 + 直接验证)** . 用代理模型给出边界与成本排序, 取成本最低的若干候选点, 用 6000 次试验直接验证; 若验证不达标则向上补 N_B 直到真正可行。最终结论以直接验证的频率为准, 代理模型只用于缩小搜索范围。

7.2 结果

阶段 A：概率曲面. 7×6 网格 ($N_A \in [150, 560]$, $N_B \in [0, 3200]$) 各 700 次试验, 代理模型对网格点的最大绝对残差为 0.036, 足以用来定位边界。网格本身已经说明了两种介质的分工:

表 7 (N_A, N_B) 网格上的导通概率与成本 (节选, $T = 700$)

$N_A \setminus N_B$	0	300	700	1200	2000	3200
250	0.034	0.080	0.074	0.149	0.269	0.583
380	0.290	0.397	0.479	0.643	0.879	0.990
440	0.550	0.624	0.774	0.879	0.976	1.000
500	0.779	0.864	0.951	0.990	0.999	1.000
560	0.917	0.950	0.993	—	—	—

阶段 B、C：沿边界搜索与验证. 代理模型给出的可行边界上, 成本最低的五个候选依次是 (552, 0)、(512, 408)、(472, 826)、(432, 1301)、(592, 0), 对应成本 8.194、8.284、8.390、8.593、8.788 元。由于阶段 C 只会把 N_B 往上补 (补到真正可行为止), 每个候选的**实测成本只会不小于上表**, 因此一旦排在最前的候选通过验证, 它就是全局最优。6000 次试验的直接验证:

表 8 候选点的大样本验证 ($T = 6000$)

N_A	N_B	导通概率 $\hat{P}$	95% 置信区	成本 / 元	是否可行
			间		
552	0	0.9108	(0.9034, 0.9178)	8.1939	是
512	408	0.8992	(0.8913, 0.9065)	8.2838	否 (未达 0.90)
512	468	0.9137	(0.9063, 0.9205)	8.3843	是, 但更贵
472	826	0.8923	(0.8842, 0.8999)	8.3904	否

表 8 中未通过三个探测点最初只留在运行日志里。为使表格可溯源,

p4_backfill_probes.py 用同一随机种子与同样的 6000 次试验把它们重算了一遍, 得到的 $\hat{P}$ 与当初逐位相同 (0.8992 / 0.8923 / 0.8885) ——这同时也验证了整套仿真的可复现性: 随机数由 SeedSequence 按 worker 分叉, 结果与并行度和运行时刻都无关。

答案：最优填充量为 $N_A = 552$ 根介质 A、 $N_B = 0$ 颗介质 B，即 $\varphi_A = 0.78\%$ 、 $\varphi_B = 0$ ，导通概率 0.9108，总成本 8.1939 元。也就是说，在题目给定的价格下，掺入介质 B 并不划算，最低成本方案就是问题三的纯 A 方案。

把阶段 A 的网格画成概率曲面即图 7。红线是 $P = 90\%$ 的可行边界，白线是等成本线（斜率由 c_A/c_B 决定）；两族线在 $N_B = 0$ 处相切——这正是“最优解落在边界端点”的几何含义。空心圆是阶段 C 直接验证过的边界点，星号是最优解。

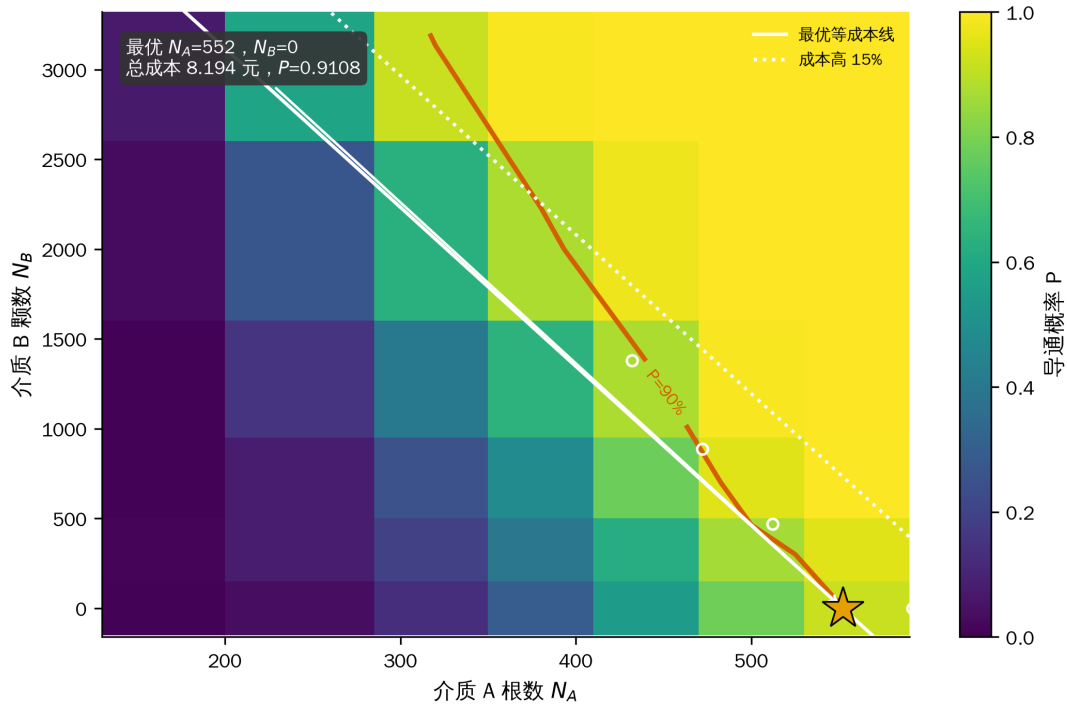


图 7 (N_A, N_B) 平面上的导通概率、可行边界与等成本线

为什么便宜的介质 B 反而用不上？关键不是单价，而是每元钱换来多少导通概率。把网格上两种介质的边际收益都折算成“每元的 ΔP ”（ $N_B = 0$ 处的差分， $T = 700$ ，故有约 ± 0.02 的抽样噪声）：

表 9 两种介质的边际效率 (ΔP / 元)

N_A	150	250	320	380	440	500	560
纯 A 时的 P	0.006	0.034	0.130	0.290	0.550	0.779	0.917
加介质 A	—	0.019	0.092	0.180	0.292	0.257	0.156
加介质 B	0.006	0.091	0.122	0.213	0.148	0.171	0.065

两条效率曲线在 $N_A \approx 400$ 附近交叉：介质稀疏时 B 更划算，接近渗流阈值时 A 更划算。机理是清楚的——半径 200 nm 的球只能把相距不到约 464 nm 的两根棒接

起来，它的作用是”补桥”；而长 5000 nm 的棒本身就跨越半个微构体，在阈值附近每多一根，贯通团簇的生长是级联式的，边际收益陡得多。

而 $P \geq 0.90$ 这个约束恰恰把可行解全部推到交叉点的右侧：即使掺入 3200 颗 B ($\varphi_B = 10.7\%$ ，成本 5.36 元)， $N_A = 380$ 时也只能到 0.990，成本已达 11.0 元。可行域内 A 始终占优，于是最优解退化到边界的端点 $N_B = 0$ 。

这个结论对价格是敏感的，而且敏感点可以算出来。设可行边界为 $N_B = g(N_A)$ ，则边界上的成本 $C(N_A) = c_A N_A + c_B g(N_A)$ ， $dC/dN_A = c_A + c_B g'$ 。 $g' < 0$ ，故当 $c_B < -c_A/g'$ 时成本随 N_A 增大而上升，此时应当多用 B。盈亏平衡价为 $c_B^* = -c_A/g'$ 。第 8.2 节由已验证的边界点实测得 $g' = -11.39$ 颗/根，对应 $c_B^* = 0.0389$ 元/ μm^3 ——现价 0.05 元/ μm^3 比它高 28%，因此最优解落在 $N_B = 0$ 。

八、模型检验、灵敏度与稳健性

8.1 几何内核的独立校验

论文全部结论建立在两个原语上：线段最短距离与边界截断。这两个错了，后面所有概率都是错的，而且错得很隐蔽。因此在跑任何仿真之前先把它钉死 (src/validate.py, 全部通过)：

表 10 几何内核校验

编号	检验内容	结果
T1	线段最短距离解析解 vs 900×900 暴力采样	最大超出量 2.3×10^{-13} nm
T2	题面边界截断图例的算例逐坐标复核	越界段 (5000, 6000) $\rightarrow$ (-5000, -4000)，与题面的边界截断图例一致
T3	附件往返测试 ：把碎片还原为母介质轴线后重新截断	组 1/2/3 共 334/334 根完整介质逐坐标一致
T4	球柱体近似（假设 A1）的影响	临界接触中受端帽形状影响者约 3.3%
T5	导通判定逻辑（贯通/断开/1 nm 间隙/单边/碎片开关）	5 项全部符合预期
T6	分块加速的接触检索 vs 朴素两两比较	三种规模下 逐位一致

T3 最有分量：它同时验证了截断实现、周期盒尺寸判断和碎片还原三件事。若第 2.2 节对组 1、组 2 周期盒的判断有误，T3 不可能对 29 根介质逐坐标复现。

8.2 灵敏度分析

全部固定在 $\varphi = 0.70\%$ ($N_A = 495$) ——渗流跃变最陡的位置，对任何扰动都最敏感。

S1 导通判据阈值 δ . $\delta = 1.8 \text{ nm}$ 是题目给定的，扫描它是为了说明结论不是卡在阈值的某个巧合上 ($T = 2000$):

表 11 导通概率对判据阈值的敏感性

δ / nm	0.90	1.35	1.80	2.25	2.70
$\hat{P}$	0.7130	0.7240	0.7370	0.7495	0.7610

δ 在 $\pm 50\%$ 范围内变动，导通概率只在 0.713–0.761 之间移动，极差 0.048。这是合理的： $\delta = 1.8 \text{ nm}$ 相对介质尺度 ($r = 30$ 、 $h = 5000$) 小两到三个数量级，它只微调”接触”的判定边界，不改变团簇结构。

S2 建模口径. 同一体积分数下，把各条口径分别换掉 ($T = 4000$):

表 12 建模口径对导通概率的影响

口径	$\hat{P}$	95% 置信区间	相对基准
基准：附件标定方向	0.7418	(0.7280, 0.7551)	—
+ 碎片各自独立			
方向改为球面均匀	0.4940	(0.4785, 0.5095)	−0.248
碎片粘合为同一导体	1.0000	(0.9990, 1.0000)	+0.258
(已否决)			

这张表是本文最重要的一张灵敏度表，它给出两条明确结论：

1. **方向分布的影响 (0.248) 是判据阈值影响 (0.048) 的 5 倍以上。**也就是说，本文结论的不确定性几乎全部来自”题面的’任意方向’该怎么读”，而不是来自数值精度或题目给定的物理参数。这也是第 9.2 节把它列为首要局限的原因。
2. **“碎片粘合”口径把概率顶到 1.0000 (8000 次试验中 8000 次导通)。**这正是假设 A2 被否决的量化理由：在该口径下问题二至四全部退化为平凡问题，任何体积分数都必然导通。

S3 介质 B 的盈亏平衡价. 问题四的答案是”不掺 B”，但这是**价格**的结论而非几何的结论，因此必须回答：B 便宜到什么程度才值得掺？

取阶段 C 中已用 6000 次试验验证过 $P \geq 0.90$ 的边界点 (432, 1379)、(472, 886)、(512, 468)、(552, 0) 作线性拟合，得可行边界斜率

$$g' = \frac{dN_B}{dN_A} = -11.39 \text{ 颗/根},$$

即每少用 1 根介质 A，需要补约 11.4 颗介质 B 才能守住 90% 的导通概率。代入 $c_B^* = -c_A/g'$ 得盈亏平衡单颗成本 1.304×10^{-3} 元，换算为单价

$$c_B^*/(V_B/10^9) = 0.0389 \text{ 元}/\mu\text{m}^3.$$

现价是 $0.05 \text{ 元}/\mu\text{m}^3$ ，比盈亏平衡价高 28%（等价地，需再降价 22.2%）。这解释了为什么最优解干脆地落在 $N_B = 0$ 而不是某个“少量掺一点”的中间配比：当前价格并不处在临界附近。

需要说明的是， $N_A > 552$ 的区段上 N_B 已经取到 0，那里的点是“配足过头”的内点而不是边界点；把它们放进拟合会把斜率压平到 -9.11 、把盈亏平衡价抬到 $0.0486 \text{ 元}/\mu\text{m}^3$ ，从而错误地得出“现价已接近临界”的结论。

图 8 把这两件事画在同一量程（0–1）上对照：左图的 δ 扫描几乎是一条水平线，右图的口径差异则跨越大半个坐标轴。两图刻意不放大纵轴——把 0.048 的变化画进 0.70–0.78 的窄区间会显得陡峭，与“对阈值不敏感”这个结论相反。

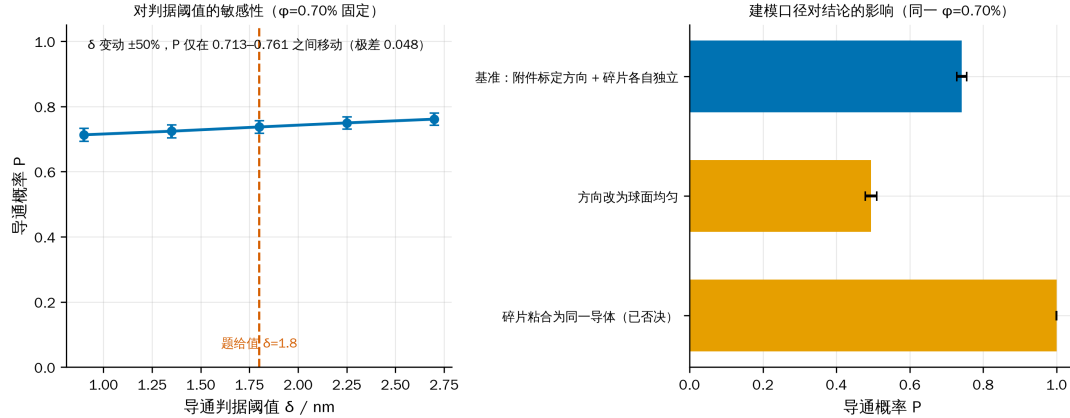


图 8 导通概率对判断阈值与建模口径的敏感性

8.3 蒙特卡洛收敛性

同一配置 ($\varphi = 0.70\%$) 按不同试验次数重复估计，Wilson 区间半宽随 $1/\sqrt{T}$ 收窄：

表 13 蒙特卡洛的收敛性

试验次数 T	250	500	1000	2000	4000	8000
$\hat{P}$	0.7320	0.7580	0.7700	0.7535	0.7592	0.7546

试验次数 T	250	500	1000	2000	4000	8000
区间半宽	0.0546	0.0374	0.0261	0.0189	0.0132	0.0094

估计值在 $T \geq 2000$ 之后稳定在 0.754–0.759，与问题二用 $T = 8000$ 得到的 0.7458 相差 0.009，与区间半宽同量级（两次估计用了不同的随机种子）。图 9 把这两件事分开画：左图是估计值随样本量趋稳的过程，右图在双对数坐标下把区间半宽与 $1/\sqrt{T}$ 参考线对照，两者平行，说明误差确实按蒙特卡洛的标准速率收窄，没有出现相关样本导致的收敛变慢。

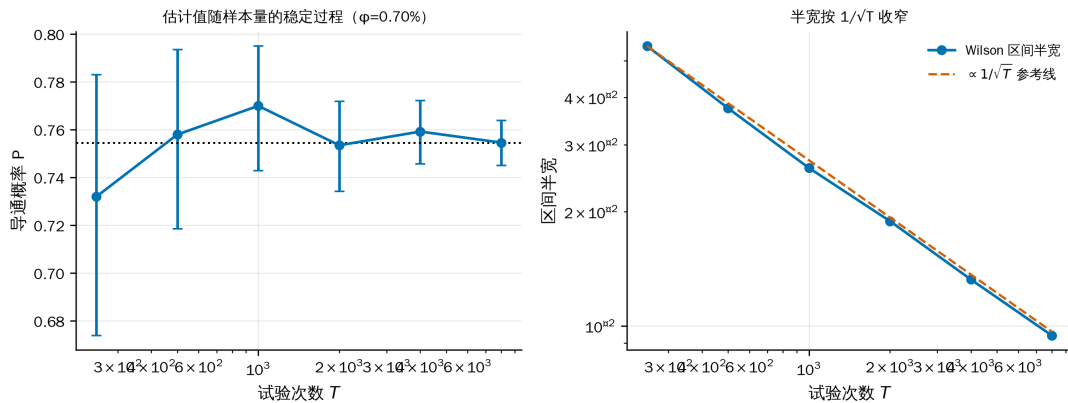


图 9 蒙特卡洛估计的收敛性

样本量是按分辨率需要定的，不是越大越好：问题三要在 0.01% 的体积分数网格上分辨相邻点，相邻点相差约 7 根介质、对应 $\Delta P \approx 0.02$ ，因此取 $T = 4000$ （半宽约 0.009）即可分开，最终答案再用 $T = 12000$ 复核；问题四的网格只用于缩小范围，取 $T = 700$ 就够，候选点才用 $T = 6000$ 直接验证。

九、模型评价与推广

9.1 优点

1. 判定内核经过独立校验，而非“看起来对”。尤其是 T3 的附件往返测试，用真实数据把截断规则、周期盒尺寸和碎片还原三件事一次性钉死。
2. 建模口径由数据定，不由猜测定。方向分布、周期盒、碎片口径三条都做了检验；其中方向分布的各向异性若被忽略，问题二的概率会系统性偏低近一半。
3. 问题四利用了单调性这一结构性质，把二维整数搜索严格地降为沿可行边界的一维搜索，并且最终结论由直接仿真验证，代理模型只承担缩小范围的职责。
4. 所有数字可溯源。论文中每个数值都来自 `results/*.json`，由脚本一次跑出。

9.2 缺点与局限

1. **方向分布的口径不确定性没有被消除，只是被量化了。**题面只说”任意方向”，附件的分布与球面均匀差异显著。本文并列给出两套答案，但无法断定组委会的参考答案取哪一套；这是本文最大的不确定来源，其影响远大于所有数值误差之和。
2. **球柱体近似（假设 A1）在端帽附近不精确。**T4 估计临界接触中约 3.3% 受此影响，方向是**略微高估**接触，因而略微高估导通概率。
3. **介质 B 的越界处理是近似的，且对照的结果与直觉相反。**被边界切开的球碎片按整球处理（`sphere_mode="wrap"`）。对照口径是把球心限制在盒内 R 深处、保证整球不越界（`sphere_mode="inside"`）。实测（ $T = 4000$ ）：

配置	wrap	inside	差值
$N_A = 440, N_B = 1200$	0.8878	0.9100	-0.0222
$N_A = 500, N_B = 700$	0.9375	0.9483	-0.0108

本来预期”整球近似会高估跨接能力”，实际却是 wrap 更低。原因不在端帽近似，而在**有效数密度**：inside 把同样多的球塞进 $(1 - 2R/L)^3 = 88.5\%$ 的体积里，内部密度反而更高。两者相差 0.01-0.02，小于方向口径的影响（0.248）一个数量级。问题四的最优解 $N_B = 0$ ，这条假设**不影响最终答案**，只影响可行边界的位置。

4. **解析对照只做到量级，不能替代仿真。**第 5.2 节用排除体积判据给出了独立的阈值估计（ $\varphi_c \approx 0.777\%$ ），但该判据假定无限大体系且各向同性，与本题”棒长恰为盒边一半、方向有偏”的设定相差正是 $0.777\% \rightarrow 0.705\% \rightarrow 0.607\%$ 这两步。因此它只能用来核对仿真没有系统性跑偏，无法脱离仿真去预测其它几何参数下的阈值。
5. **蒙特卡洛给的是频率而非精确值。**所有概率都带 Wilson 区间；问题三的答案 0.78% 依赖于 0.77% 与 0.78% 两格能被区分开，本文用 $T = 4000$ 与 $T = 12000$ 两次独立估计确认了这一点，但若组委会的参考答案用了不同的生成器细节，相邻格的归属仍可能改变。
6. **组 1、组 2 的几何与题面不符**，本文按数据实际几何判定。若组委会本意是 10000^3 立方体，则这两组的输入本身有误，需要以官方勘误为准。

9.3 推广

判定内核与题目的具体形状无关：把”球柱体 + 球”换成任意可计算最短距离的凸体，整套流程（截断 → 接触图 → 并查集 → 蒙特卡洛 → 仿真优化）可直接迁移到碳纳米管/石墨烯片复合材料的导电阈值预测、纤维增强材料的逾渗设计等场景。问题四的”单调约束 + 线性成本 ⇒ 最优解在可行边界上”这一论证，对任何”加料只会更容易达标”的配方优化都成立。

十、结论

表 14 四问结论汇总（主口径：附件标定的方向分布）

问题	答案
一	组 1 不导通；组 2 导通；组 3 导通
二	$\varphi = 0.50\%$: $P = 0.2193$; 0.60% : 0.4798 ; 0.70% : 0.7458 ; 1.00% : 0.9994
三	使 $P \geq 90\%$ 的最低体积分数 $\varphi = \mathbf{0.78\%}$ (552 根介质 A)
四	最优填充量 $N_A = 552$ 、 $N_B = 0$ ，即 $\varphi_A = 0.78\%$ 、 $\varphi_B = 0$ ，总成本 8.19 元

三点值得单独指出：

1. **问题一的关键不在判定，而在还原。**附件的一行是边界截断后的碎片而非一根介质；组 1、组 2 的周期盒也不是立方体。不做这一步，三个微构体的体积分数会分别错到 1/100，判定的机理解释也无从谈起。
2. **导通是跃变而非线性响应，且跃变发生在巨团簇形成之前。**体积分数从 0.50% 到 1.00%，导通概率由 0.22 升到 0.999，而最大团簇始终只占约 2%–22% 的碎片（第 5.3 节）。贯通靠的是 2–3 根介质接力的短链，不是吞掉大半介质的巨团簇。工程上这意味着两件事：填充量设计必须留出跃变宽度的余量；本题的阈值也不能直接外推到更大的微构体——盒子变大而介质尺寸不变时，阈值会升高。
3. **问题四的答案是”不掺 B”，但这是价格的结论而不是几何的结论。**介质 B 在介质稀疏时每元的边际收益高于 A，只是 $P \geq 90\%$ 的约束把可行解推到了 A 占优的区段。B 的单价只要降到 0.0389 元/ μm^3 以下（现价再降 22.2%），最优解就会变成混填。

全文的最大不确定性不在数值精度，而在方向分布这一建模口径：若按题面字面取球面均匀，问题二的四个概率变为 0.0874 / 0.2325 / 0.4911 / 0.9940，问题三的答

案变为 0.86%。两套结果全程并列给出，读者可按对题面的理解自行取用。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码实现与调试、图表版式检查和文字校对，详细使用情况见支撑材料。

说明：本目录为赛后复现，非赛内提交稿。上述声明沿用 CUMCM《人工智能工具使用规定》的二选一句式；“华数杯”竞赛的 AI 披露要求以其官方章程为准，正式提交前须按主办方当年规定核对声明位置与措辞。

参考文献

[1] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 5 版. 北京: 高等教育出版社, 2018.

[2] 司守奎, 孙兆亮. 数学建模算法与应用 [M]. 2 版. 北京: 国防工业出版社, 2015.

[3] 徐钟济. 蒙特卡罗方法 [M]. 上海: 上海科学技术出版社, 1985.

[4] 盛骤, 谢式千, 潘承毅. 概率论与数理统计 [M]. 4 版. 北京: 高等教育出版社, 2008.

[5] 李文春, 沈烈, 孙晋, 郑强, 章明秋. 多壁碳纳米管填充高密度聚乙烯复合材料的导电性和动态流变行为 [J]. 高分子学报, 2006(2): 269-273.

[6] BALBERG I, ANDERSON C H, ALEXANDER S, et al. Excluded volume and its relation to the onset of percolation[J]. Physical Review B, 1984, 30(7): 3933-3943.

[7] ERICSON C. Real-Time Collision Detection[M]. San Francisco: Morgan Kaufmann, 2005.

附录

附录 A 支撑材料文件列表

文件	内容	对应论文位置
src/microstructure.py	几何与渗流内核	第 3 节
src/load_attachment.py	附件读取与母介质还原	第 2.1 节
src/audit_attachment.py	数据审计与 KS 检验	第 2 节、图 2、图 3
src/validate.py	几何内核 6 项校验	第 8.1 节表 10
src/simulate.py	蒙特卡洛驱动（并行、Wilson 区间、可复现种子）	第 5.1 节

文件	内容	对应论文位置
src/solve_p1.py	问题一判定	第 4 节表 2、图 5
src/solve_p2.py	问题二概率估计	第 5 节、图 6
src/solve_p3.py	问题三阈值反解	第 6 节
src/solve_p4.py	问题四成本优化	第 7 节表 7、表 8、图 7
src/p4_backfill_probes.py	补记阶段 C 未通过的探测点 (同种子可复现)	第 7.2 节表 8
src/p4_break_even.py	由已验证边界点求边界斜率 与介质 B 盈亏平衡价	第 8.2 节 S3
src/sensitivity.py	灵敏度与收敛性	第 8.2、8.3 节、图 8、图 9
src/sphere_mode_check.py	介质 B 越界处理口径的对照	第 9.2 节
src/theory_check.py	排除体积判据、跃变中点、 边际效率	第 5.2、7.2 节
src/cluster_stats.py	最大团簇占比 (渗流序参量)	第 5.3 节、图 10
src/make_results_bundle.py	合并结果文件并装配附录 B 源程序	附录 B
src/paper_figures.py	全部正文图件	图 1–图 10
results/*.json	论文中每个数字的出处	全文
figures_paper/图注清单.md	图注与生成脚本对照	全文
AI 工具使用详情.pdf	AI 工具使用详情 (工具与版 本、用途环节、使用过程、采 纳与核验)	AI 工具使用声明

附录 B 完整可运行源程序

本附录由 `src/make_results_bundle.py` 从 `src/` 直接装配，与实际运行的代码逐字一致。运行顺序见附录 A 之后的说明。依赖 Python 3.11 与 `numpy` / `scipy` / `matplotlib` / `openpyxl`。

`src/microstructure.py`

```
#!/usr/bin/env python3
```

```
""" 微结构导电仿真的几何与渗流内核。
```

本模块只做三件事，不涉及任何题目分支逻辑：

1. **边界截断** **: 把一根越界的介质按题目给定的规则平移回微构体，得到若干碎片；
2. **接触判定** **: 两个介质 (或介质与带电面) 之间的最短表面距离是否不超过 1.8 nm ;
3. **导通判定** **: 并查集把接触关系连成连通块，判断是否存在连接左右带电面的通路。

几何约定

```

-----

介质 A 建模为 ** 球柱体 (spherocylinder / capsule) **: 半径  $r=30\text{ nm}$  的圆柱两端各加一个
半球帽。这样两个介质 A 的表面最短距离就精确等于两条轴线段的最短距离减去  $2r$ ,
判定可以完全向量化。题目里的介质 A 是平端面直圆柱, 与球柱体的差别只出现在
端面附近, 且体积仅相差  $(4/3) r^3 / (r^2 h) = 4r/(3h) = 0.8\%$ 。
`validate.py` 里量化了这一近似对导通判定的影响。

介质 B 是半径  $200\text{ nm}$  的正球体, 本身就是球, 无需近似。
"""

from __future__ import annotations

import numpy as np

# ----- 物理常数

EDGE = 10000.0          # 微构体边长 (nm)
R_A = 30.0              # 介质 A 底面半径 (nm)
H_A = 5000.0            # 介质 A 高 (nm)
R_B = 200.0             # 介质 B 半径 (nm)
GAP = 1.8               # 导通判定的最大表面间距 (nm)

V_BOX = EDGE ** 3        #  $1.0e12\text{ nm}^3$ 
V_A = np.pi * R_A ** 2 * H_A    #  $1.41372e7\text{ nm}^3$ 
V_B = 4.0 / 3.0 * np.pi * R_B ** 3    #  $3.35103e7\text{ nm}^3$ 

# 成本: 介质 A  $1.05\text{ 元}/m^3$ , 介质 B  $0.05\text{ 元}/m^3$ ;  $1\text{ m}^3 = 1e9\text{ nm}^3$ 
COST_A = 1.05 * V_A / 1e9        # 元 / 根
COST_B = 0.05 * V_B / 1e9        # 元 / 颗

# ----- 边界截断

def wrap_segment(p: np.ndarray, q: np.ndarray, box: np.ndarray) -> list[tuple[np.ndarray, np.ndarray]]:
    """ 把轴线段  $p \rightarrow q$  按边界截断规则折回盒子, 返回若干条完全位于盒内的子段。

    做法: 把线段参数化为  $t \in [0, 1]$ , 在每个坐标轴上求出它穿越周期格点边界的所有  $t$ ,
    以这些  $t$  为断点切段; 每一小段整体位于同一个周期像里, 平移回来即可。
    这与题面 "把超出部分沿越界方向的反方向平移一个边长" 逐字等价, 并且天然支持
    需要多次平移的情形。
    """
    half = box / 2.0
    d = q - p
    ts = [0.0, 1.0]
    for j in range(3):
        if abs(d[j]) < 1e-12:
            continue
        # 穿越平面  $x_j = \text{half}_j + k \cdot \text{box}_j$ 
        k_lo = np.floor((min(p[j], q[j]) - half[j]) / box[j])
        k_hi = np.ceil((max(p[j], q[j]) - half[j]) / box[j])
        for k in range(int(k_lo), int(k_hi) + 1):
            plane = half[j] + k * box[j]
            t = (plane - p[j]) / d[j]
            if 1e-12 < t < 1 - 1e-12:
                ts.append(float(t))
    ts = sorted(set(ts))

```

```

out: list[tuple[np.ndarray, np.ndarray]] = []
for t0, t1 in zip(ts[:-1], ts[1:]):
    if t1 - t0 < 1e-12:
        continue
    a = p + t0 * d
    b = p + t1 * d
    mid = 0.5 * (a + b)
    shift = np.round(mid / box) * box          # 该子段所在周期像的偏移
    seg_a, seg_b = a - shift, b - shift
    # 数值上把端点压回盒内, 避免 ±1e-12 的溢出
    np.clip(seg_a, -half, half, out=seg_a)
    np.clip(seg_b, -half, half, out=seg_b)
    out.append((seg_a, seg_b))
return out

# ----- 距离计算

def seg_seg_dist(p1: np.ndarray, q1: np.ndarray, p2: np.ndarray, q2: np.ndarray) -> np.ndarray:
    """ 成对线段最短距离, 全向量化。

    p1/q1 形状 (n,1,3), p2/q2 形状 (1,m,3) 时返回 (n,m)。
    算法是 Ericson 《Real-Time Collision Detection》的 ClosestPtSegmentSegment,
    分支用 np.where 展开; 退化 (零长) 线段按点处理。
    """
    d1 = q1 - p1
    d2 = q2 - p2
    r = p1 - p2

    a = np.sum(d1 * d1, axis=-1)
    e = np.sum(d2 * d2, axis=-1)
    f = np.sum(d2 * r, axis=-1)
    c = np.sum(d1 * r, axis=-1)
    b = np.sum(d1 * d2, axis=-1)

    eps = 1e-12
    a_safe = np.maximum(a, eps)
    e_safe = np.maximum(e, eps)

    denom = a * e - b * b
    s = np.where(denom > eps, (b * f - c * e) / np.where(denom > eps, denom, 1.0), 0.0)
    s = np.clip(s, 0.0, 1.0)

    t = (b * s + f) / e_safe

    # t 越界时钳位并回代求 s
    s_lo = np.clip(-c / a_safe, 0.0, 1.0)
    s_hi = np.clip((b - c) / a_safe, 0.0, 1.0)
    s = np.where(t < 0.0, s_lo, np.where(t > 1.0, s_hi, s))
    t = np.clip(t, 0.0, 1.0)

    # 退化线段
    s = np.where(a <= eps, 0.0, s)
    t = np.where(e <= eps, 0.0, np.where(a <= eps, np.clip(f / e_safe, 0.0, 1.0), t))

```

```

diff = r + s[..., None] * d1 - t[..., None] * d2
return np.sqrt(np.maximum(np.sum(diff * diff, axis=-1), 0.0))

def contact_pairs(p: np.ndarray, q: np.ndarray, thresh: float, block: int = 256) -> np.ndarray:
    """ 返回所有轴线距离 thresh 的线段对下标 (k,2)。

    分块 + 轴对齐包围盒预筛。朴素写法要一次性开出  $M^2$  的十几个临时数组，
     $M1300$  时光是分配和读写就占了九成时间；分块后临时数组常驻缓存，
    实测在  $\approx 1\%$  规模上快 6 倍以上，结果与朴素写法逐位相同 (validate.py T6)。
    """
    m = len(p)
    lo = np.minimum(p, q) - thresh
    hi = np.maximum(p, q)
    out: list[np.ndarray] = []
    for s in range(0, m, block):
        e = min(s + block, m)
        # 只与下标更大的比较，避免重复
        cand = np.nonzero(np.all(lo[None, s:e, :] <= hi[s:, None, :], axis=-1)
                           & np.all(lo[s:, None, :] <= hi[None, s:e, :], axis=-1))
        rows = cand[0] + s          # 全局下标 j
        cols = cand[1] + s          # 全局下标 i
        keep = rows > cols
        rows, cols = rows[keep], cols[keep]
        if rows.size == 0:
            continue
        d = seg_seg_dist(p[cols], q[cols], p[rows], q[rows])
        hit = d <= thresh
        if np.any(hit):
            out.append(np.stack([cols[hit], rows[hit]], axis=1))
    return np.concatenate(out) if out else np.zeros((0, 2), dtype=int)

def point_seg_dist(pts: np.ndarray, p: np.ndarray, q: np.ndarray) -> np.ndarray:
    """ 点到线段的距离。 pts (n,1,3), p/q (1,m,3) -> (n,m)。 """
    d = q - p
    w = pts - p
    dd = np.maximum(np.sum(d * d, axis=-1), 1e-12)
    t = np.clip(np.sum(w * d, axis=-1) / dd, 0.0, 1.0)
    diff = w - t[..., None] * d
    return np.sqrt(np.maximum(np.sum(diff * diff, axis=-1), 0.0))

# ----- 并查集

class DSU:
    __slots__ = ("parent",)

    def __init__(self, n: int) -> None:
        self.parent = list(range(n))

    def find(self, x: int) -> int:
        p = self.parent
        while p[x] != x:
            p[x] = p[p[x]]
        x = p[x]

```

```

        return x

def union(self, x: int, y: int) -> None:
    rx, ry = self.find(x), self.find(y)
    if rx != ry:
        self.parent[ry] = rx

# ----- 生成配置

def sample_orientations(n: int, rng: np.random.Generator, mode: str = "polar_uniform") -> np.ndarray:
    """ 生成  $n$  个单位方向向量。

    mode="isotropic" : 球面均匀, 即  $|u_x| \sim U(0,1)$ 。这是 "任意方向、不考虑重力"
                        最自然的物理读法。
    mode="polar_uniform" :  $\sim U(0,1)$ ,  $\sim U(0,2)$ , 以  $**X$  轴 (带电面法向)  $**$  为极轴,
                         $u = (\cos \theta, \sin \theta \cos \phi, \sin \theta \sin \phi)$ 。这是附件数据实际服从的分布:
                        对组 3 的 354 根母介质做 KS 检验, 各向同性被拒绝 ( $D=0.243$ ,
                         $p=6.7e-19$ ), 本分布不被拒绝 ( $D=0.036$ ,  $p=0.73$ )。
                        它在两极堆积, 使介质明显偏向带电面法向, 从而显著提高导电概率。

    """
    if mode == "isotropic":
        u = rng.normal(size=(n, 3))
        return u / np.linalg.norm(u, axis=1, keepdims=True)
    if mode == "polar_uniform":
        theta = rng.uniform(0.0, np.pi, n)
        phi = rng.uniform(0.0, 2 * np.pi, n)
        s = np.sin(theta)
        return np.stack([np.cos(theta), s * np.cos(phi), s * np.sin(phi)], axis=1)
    raise ValueError(f"未知方向分布: {mode}")

def sample_rods(n: int, rng: np.random.Generator, box: np.ndarray,
                length: float = H_A,
                orientation: str = "polar_uniform") -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """ 随机生成  $n$  根介质  $A$  并做边界截断。

    中心在盒内均匀; 方向分布见 ``sample_orientations``。
    返回碎片起点、终点, 以及每个碎片所属的母介质编号。

    """
    centers = (rng.random((n, 3)) - 0.5) * box
    u = sample_orientations(n, rng, orientation)
    p = centers - 0.5 * length * u
    q = centers + 0.5 * length * u

    starts: list[np.ndarray] = []
    ends: list[np.ndarray] = []
    owner: list[int] = []
    for i in range(n):
        for a, b in wrap_segment(p[i], q[i], box):
            starts.append(a)
            ends.append(b)
            owner.append(i)
    if not starts:
        empty = np.zeros((0, 3))
        return empty, empty, np.zeros(0, dtype=int)

```

```

return np.array(starts), np.array(ends), np.array(owner, dtype=int)

def sample_spheres(n: int, rng: np.random.Generator, box: np.ndarray,
                  mode: str = "wrap") -> tuple[np.ndarray, np.ndarray]:
    """ 随机生成  $n$  颗介质  $B$ 。

    mode="wrap" : 中心在盒内均匀, 越界部分按截断规则折回, 碎片以其所在周期像的
                  球心表示 (碎片是球与盒的交, 用整球近似, 只在距壁 200 nm 内有偏差);
    mode="inside" : 把球心限制在  $[-(L/2-R), L/2-R]$ , 保证整颗球都在盒内, 不产生碎片。
    两种取法在灵敏度分析里对比。
    """
    if mode == "inside":
        c = (rng.random((n, 3)) - 0.5) * (box - 2 * R_B)
        return c, np.arange(n)

    c = (rng.random((n, 3)) - 0.5) * box
    half = box / 2.0
    centers: list[np.ndarray] = []
    owner: list[int] = []
    for i in range(n):
        # 每个越界方向都产生一个镜像碎片
        offs = [np.zeros(3)]
        for j in range(3):
            if c[i, j] + R_B > half[j]:
                offs = offs + [o + np.eye(3)[j] * -box[j] for o in offs]
            elif c[i, j] - R_B < -half[j]:
                offs = offs + [o + np.eye(3)[j] * box[j] for o in offs]
        for o in offs:
            centers.append(c[i] + o)
            owner.append(i)
    return np.array(centers), np.array(owner, dtype=int)

# ----- 导通判定

def percolates(seg_p: np.ndarray, seg_q: np.ndarray,
              sph_c: np.ndarray | None = None,
              owner_rod: np.ndarray | None = None,
              owner_sph: np.ndarray | None = None,
              bond_fragments: bool = False,
              edge: float = EDGE) -> bool:
    """ 判断微构体是否导通 (左右带电面之间存在导电通路)。

    bond_fragments=False (默认): 边界截断产生的每个碎片是独立导体;
    bond_fragments=True : 同一根母介质的所有碎片电学上视为一体 (灵敏度对照)。
    """
    half = edge / 2.0
    n_rod = len(seg_p)
    n_sph = 0 if sph_c is None else len(sph_c)
    n = n_rod + n_sph
    if n == 0:
        return False

    dsu = DSU(n + 2)
    LEFT, RIGHT = n, n + 1

```

```

# --- 介质与带电面
if n_rod:
    lo = np.minimum(seg_p[:, 0], seg_q[:, 0]) - R_A
    hi = np.maximum(seg_p[:, 0], seg_q[:, 0]) + R_A
    for i in np.nonzero(lo <= -half + GAP)[0]:
        dsu.union(LEFT, int(i))
    for i in np.nonzero(hi >= half - GAP)[0]:
        dsu.union(RIGHT, int(i))
if n_sph:
    for i in np.nonzero(sph_c[:, 0] - R_B <= -half + GAP)[0]:
        dsu.union(LEFT, n_rod + int(i))
    for i in np.nonzero(sph_c[:, 0] + R_B >= half - GAP)[0]:
        dsu.union(RIGHT, n_rod + int(i))

# --- 棒-棒
if n_rod > 1:
    for a, b in contact_pairs(seg_p, seg_q, 2 * R_A + GAP):
        dsu.union(int(a), int(b))

# --- 球-球 与 球-棒 (介质 B 数量可达上万, 用 KD 树做邻域检索)
if n_sph:
    from scipy.spatial import cKDTree
    tree = cKDTree(sph_c)
    if n_sph > 1:
        for a, b in tree.query_pairs(2 * R_B + GAP, output_type="ndarray"):
            dsu.union(n_rod + int(a), n_rod + int(b))
    if n_rod:
        # 沿每根碎片轴线按 STEP 采样, 查半径 = 接触半径 + 半个采样步长,
        # 保证不漏掉任何真实接触; 再用精确的点—线段距离过滤候选。
        reach = R_A + R_B + GAP
        step = 200.0
        samples, tags = [], []
        for i in range(n_rod):
            a, b = seg_p[i], seg_q[i]
            L = float(np.linalg.norm(b - a))
            k = max(2, int(np.ceil(L / step)) + 1)
            ts = np.linspace(0.0, 1.0, k)[: , None]
            samples.append(a + ts * (b - a))
            tags.append(np.full(k, i, dtype=int))
        pts = np.concatenate(samples)
        tags = np.concatenate(tags)
        radius = reach + step / 2.0 + 1e-9
        for pt_idx, sph_list in enumerate(tree.query_ball_point(pts, radius)):
            if not sph_list:
                continue
            rod = int(tags[pt_idx])
            cand = np.asarray(sph_list, dtype=int)
            d = point_seg_dist(sph_c[cand][: , None, :],
                               seg_p[rod][None, None, :], seg_q[rod][None, None, :])[: , 0]
            for s in cand[d <= reach]:
                dsu.union(n_rod + int(s), rod)

# --- 灵敏度对照: 同母介质的碎片粘合
if bond_fragments:
    for owner, base, cnt in ((owner_rod, 0, n_rod), (owner_sph, n_rod, n_sph)):

```

```

        if owner is None or cnt == 0:
            continue
        first: dict[int, int] = {}
        for idx in range(cnt):
            o = int(owner[idx])
            if o in first:
                dsu.union(first[o], base + idx)
            else:
                first[o] = base + idx

    return dsu.find(LEFT) == dsu.find(RIGHT)

def n_rods_for_fraction(phi: float) -> int:
    """ 体积分数 -> 介质 A 根数 (题目要求四舍五入)。 """
    return int(round(phi * V_BOX / V_A))

```

src/load_attachment.py

```

#!/usr/bin/env python3
""" 读取附件 xlsx, 并把 " 碎片 " 还原成母介质 A。

附件每一行不是一根完整的介质 A, 而是边界截断之后的一个 ** 碎片 **。
组 3 有 535 行, 但按轴向分组后只有 354 个不同方向, 正好对应 354 根介质 A
(体积分数 0.50%)。数据审计 (见 audit_attachment.py) 还发现附件系统性地
丢弃了长度小于约 500 nm 的短碎片, 因此各组碎片总长略小于 根数 × 5000。
"""

from __future__ import annotations

import math
from collections import defaultdict
from pathlib import Path

import numpy as np
import openpyxl

from microstructure import EDGE

ROOT = Path(__file__).resolve().parents[1]
ATTACHMENT = ROOT.parent / "01_ 题目与附件" / " 附件.xlsx"

# 组 1、组 2 的 y、z 坐标在 ±500 处发生周期回绕 (见 audit_attachment.py),
# 说明这两个微构体的截面是 1000×1000 nm, 只有组 3 是题面描述的 10000 立方体。
GROUP_BOX = {
    " 组 1": np.array([EDGE, 1000.0, 1000.0]),
    " 组 2": np.array([EDGE, 1000.0, 1000.0]),
    " 组 3": np.array([EDGE, EDGE, EDGE]),
}

def load_pieces(path: Path | None = None) -> dict[str, np.ndarray]:
    """ 返回 {组名: (n,6) 数组}, 每行是一个碎片的两个轴端点。 """
    wb = openpyxl.load_workbook(path or ATTACHMENT, read_only=True, data_only=True)
    out: dict[str, np.ndarray] = {}
    for name in wb.sheetnames:

```

```

        rows = [r for r in wb[name].iter_rows(values_only=True)][2:]
        vals = [[float(v) for v in r[:6]] for r in rows if r[0] is not None]
        out[name] = np.array(vals)
    return out

def _canonical_dir(p: np.ndarray, q: np.ndarray) -> tuple:
    d = q - p
    d = d / np.linalg.norm(d)
    if d[0] < 0 or (d[0] == 0 and d[1] < 0):
        d = -d
    return tuple(np.round(d, 7))

def group_by_medium(pieces: np.ndarray) -> list[list[int]]:
    """ 按轴向把碎片归到母介质。同一根介质的所有碎片方向完全相同。 """
    groups: dict[tuple, list[int]] = defaultdict(list)
    for i, row in enumerate(pieces):
        groups[_canonical_dir(row[:3], row[3:])].append(i)
    return list(groups.values())

def chain_pieces(pieces: np.ndarray, idx: list[int], box: np.ndarray) -> list[int]:
    """ 把一根母介质的碎片按首尾相接顺序排好（用于还原未截断的轴线段）。 """
    half = box / 2.0

    def on_boundary(pt):
        return [j for j in range(3) if abs(abs(pt[j]) - half[j]) < 1e-6]

    succ, pred = {}, {}
    for i in idx:
        q = pieces[i][3:]
        for j in on_boundary(q):
            tgt = q.copy()
            tgt[j] = -q[j]
            for k in idx:
                if k != i and np.allclose(pieces[k][:3], tgt, atol=1e-6):
                    succ[i], pred[k] = k, i
    heads = [i for i in idx if i not in pred]
    order = []
    for h in heads:
        cur = h
        while cur is not None and cur not in order:
            order.append(cur)
            cur = succ.get(cur)
    for i in idx:
        # 兜底：链断裂时也不丢碎片
        if i not in order:
            order.append(i)
    return order

def reconstruct_axis(pieces: np.ndarray, idx: list[int], box: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪ float]:
    """ 还原母介质未经截断的轴线段（起点、终点、已知总长）。 """
    order = chain_pieces(pieces, idx, box)
    start = pieces[order[0]][3:].copy()

```

```

total = sum(float(np.linalg.norm(pieces[i][3:] - pieces[i][:3])) for i in order)
d = pieces[order[0]][3:] - pieces[order[0]][:3]
u = d / np.linalg.norm(d)
return start, start + total * u, total

def summarize(path: Path | None = None) -> dict[str, dict]:
    """ 每组的碎片数、母介质数、总长与体积分数。"""
    data = load_pieces(path)
    out = {}
    for name, pieces in data.items():
        box = GROUP_BOX[name]
        media = group_by_medium(pieces)
        lens = [float(np.linalg.norm(r[3:] - r[:3])) for r in pieces]
        out[name] = {
            "n_pieces": len(pieces),
            "n_media": len(media),
            "total_axis_length": sum(lens),
            "implied_media_by_length": sum(lens) / 5000.0,
            "box": box.tolist(),
            "missing_length": len(media) * 5000.0 - sum(lens),
        }
    return out

if __name__ == "__main__":
    import json
    print(json.dumps(summarize(), ensure_ascii=False, indent=2))

```

src/simulate.py

```
#!/usr/bin/env python3
```

```
""" 蒙特卡洛导通概率估计 (可复现、并行)。
```

一次 " 试验 " = 随机生成一个配置 -> 边界截断 -> 建接触图 -> 判导通。
 导通概率就是伯努利参数 p , 用 *Wilson* 区间给置信区间 (样本比例接近 0 或 1 时,
 正态近似的 $p \pm 1.96 \cdot se$ 会给出越界的区间, *Wilson* 不会)。

所有随机性来自一个 *SeedSequence*, 按 *worker* 分叉, 因此结果与并行度无关,
 换机器重跑能逐位复现。

```
"""
```

```
from __future__ import annotations
```

```
import math
```

```
import os
```

```
from dataclasses import dataclass, asdict
```

```
from multiprocessing import Pool
```

```
import numpy as np
```

```
from microstructure import (EDGE, V_A, V_B, V_BOX, COST_A, COST_B,
                           percolates, sample_rods, sample_spheres)
```

```
BOX = np.full(3, EDGE)
```

```

@dataclass(frozen=True)
class Config:
    n_a: int
    n_b: int = 0
    orientation: str = "polar_uniform"
    sphere_mode: str = "wrap"
    bond_fragments: bool = False

    @property
    def phi_a(self) -> float:
        return self.n_a * V_A / V_BOX

    @property
    def phi_b(self) -> float:
        return self.n_b * V_B / V_BOX

    @property
    def cost(self) -> float:
        return self.n_a * COST_A + self.n_b * COST_B

def _run_chunk(args) -> int:
    cfg_dict, seed_state, n_trials = args
    cfg = Config(**cfg_dict)
    rng = np.random.default_rng(seed_state)
    hits = 0
    for _ in range(n_trials):
        sp, sq, own_r = sample_rods(cfg.n_a, rng, BOX, orientation=cfg.orientation)
        sc = own_s = None
        if cfg.n_b:
            sc, own_s = sample_spheres(cfg.n_b, rng, BOX, mode=cfg.sphere_mode)
        hits += bool(percolates(sp, sq, sph_c=sc, owner_rod=own_r, owner_sph=own_s,
                                bond_fragments=cfg.bond_fragments))
    return hits

def wilson(hits: int, n: int, z: float = 1.959963985) -> tuple[float, float]:
    if n == 0:
        return (0.0, 1.0)
    p = hits / n
    d = 1 + z * z / n
    c = p + z * z / (2 * n)
    h = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n))
    return ((c - h) / d, (c + h) / d)

def estimate_p(cfg: Config, trials: int, seed: int = 20260810,
               workers: int | None = None) -> dict:
    """ 返回 {p, lo, hi, hits, trials, ...}。 """
    workers = workers or min(os.cpu_count() or 1, 8)
    n_chunks = max(workers, 1)
    base = trials // n_chunks
    sizes = [base + (1 if i < trials - base * n_chunks else 0) for i in range(n_chunks)]
    sizes = [s for s in sizes if s > 0]
    seeds = np.random.SeedSequence(seed).spawn(len(sizes))

```

```

jobs = [(asdict(cfg), s, k) for s, k in zip(seeds, sizes)]

if len(jobs) == 1:
    hits = _run_chunk(jobs[0])
else:
    with Pool(len(jobs)) as pool:
        hits = sum(pool.map(_run_chunk, jobs))

lo, hi = wilson(hits, trials)
return {
    "n_a": cfg.n_a, "n_b": cfg.n_b,
    "phi_a": cfg.phi_a, "phi_b": cfg.phi_b,
    "orientation": cfg.orientation, "sphere_mode": cfg.sphere_mode,
    "bond_fragments": cfg.bond_fragments,
    "trials": trials, "hits": hits,
    "p": hits / trials, "ci_lo": lo, "ci_hi": hi,
    "half_width": (hi - lo) / 2,
    "cost_yuan": cfg.cost,
    "seed": seed,
}

def n_rods_for_phi(phi: float) -> int:
    return int(round(phi * V_BOX / V_A))

def n_spheres_for_phi(phi: float) -> int:
    return int(round(phi * V_BOX / V_B))

```

src/audit_attachment.py

```
#!/usr/bin/env python3
```

```
""" 附件数据审计。
```

在对附件做任何建模之前先回答四个问题——每一个的答案都改变了后面的模型：

- A1** 附件的一行是什么？不是一根介质 A ，而是边界截断之后的一个碎片。
- A2** 三个微构体的周期盒一样大吗？组 3 是题面的 10000° ；组 1、组 2 的 y, z 在 ± 500 处回绕，截面只有 $1000 \times 1000 \text{ nm}$ 。
- A3** 附件完整吗？不完整：长度小于约 500 nm 的短碎片被系统性丢弃。
- A4** 介质方向是各向同性的吗？不是。组 3 的方向服从“以 X 轴为极轴、 $\sim U(0,)$ ”的分布，明显偏向带电极面向；各向同性被 KS 检验以 $p \ 7e-19$ 拒绝。

A4 是全屏影响最大的一条：它把 $\approx 0.50\%$ 的导通概率抬高了约一倍。

结果写入 `results/data_audit.json`。

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from scipy import stats
```

```

from load_attachment import (GROUP_BOX, group_by_medium, load_pieces,
                             reconstruct_axis)

RESULTS = Path(__file__).resolve().parents[1] / "results"

def piece_lengths(pieces: np.ndarray) -> np.ndarray:
    return np.linalg.norm(pieces[:, 3:] - pieces[:, :3], axis=1)

def wrap_evidence(pieces: np.ndarray, box: np.ndarray) -> dict:
    """ 统计碎片端点精确落在各周期面上的次数。

    这是判断周期盒尺寸的直接证据：碎片在边界处成对出现（前一碎片终点在  $+h$ ，
    后一碎片起点在  $-h$ ），因此只要数一数端点恰好取到  $\pm h$  的次数，就能读出回绕面在哪。
    """
    half = box / 2.0
    out = {}
    for j, axis in enumerate("XYZ"):
        n = 0
        for row in pieces:
            for pt in (row[:3], row[3:]):
                if abs(abs(pt[j]) - half[j]) < 1e-6:
                    n += 1
            out[axis] = {"half_extent_nm": float(half[j]), "endpoints_on_face": int(n)}
    return out

def audit_group(name: str, pieces: np.ndarray) -> dict:
    box = GROUP_BOX[name]
    media = group_by_medium(pieces)
    lens = piece_lengths(pieces)

    # A3: 把每根母介质的碎片接起来，缺多少长度就是被丢弃的碎片
    missing = []
    for idx in media:
        _, _, total = reconstruct_axis(pieces, idx, box)
        if abs(total - 5000.0) > 1e-3:
            missing.append(5000.0 - total)

    # A4: 母介质方向的各向异性
    dirs = []
    for idx in media:
        d = pieces[idx[0]][3:] - pieces[idx[0]][:3]
        dirs.append(np.abs(d / np.linalg.norm(d)))
    dirs = np.array(dirs)

    iso = stats.kstest(dirs[:, 0], "uniform")
    polar_cdf = lambda t: np.clip((np.pi - 2 * np.arccos(np.clip(t, 0, 1))) / np.pi, 0, 1)
    pol = stats.kstest(dirs[:, 0], polar_cdf)

    return {
        "n_pieces": int(len(pieces)),
        "n_media": int(len(media)),
        "pieces_per_medium": float(len(pieces) / len(media)),
        "periodic_box_nm": box.tolist(),
    }

```

```

        "wrap_evidence": wrap_evidence(pieces, box),
        "piece_length_min_nm": float(lens.min()),
        "piece_length_max_nm": float(lens.max()),
        "total_axis_length_nm": float(lens.sum()),
        "volume_fraction": float(len(media) * np.pi * 30.0 ** 2 * 5000.0 / np.prod(box)),
        "n_media_incomplete": len(missing),
        "dropped_length_nm": float(sum(missing)),
        "dropped_fragment_len_max_nm": float(max(missing)) if missing else 0.0,
        # 每根不完整母介质缺的总长; 缺 2 个碎片的母介质会给出 >500 nm 的值,
        # 单个被丢弃的碎片本身都短于 500 nm (保留的最短碎片 526 nm)。
        "dropped_per_medium_nm": sorted(round(float(m), 1) for m in missing),
        "n_dropped_under_500": int(sum(1 for m in missing if m < 500)),
        "mean_abs_u": dirs.mean(axis=0).tolist(),
        "ks_isotropic": {"D": float(iso.statistic), "p": float(iso.pvalue)},
        "ks_polar_uniform": {"D": float(pol.statistic), "p": float(pol.pvalue)},
    }

def main() -> int:
    data = load_pieces()
    out = {
        "reference_means": {
            "isotropic": [0.5, 0.5, 0.5],
            "polar_uniform_about_x": [2 / np.pi, (2 / np.pi) ** 2, (2 / np.pi) ** 2],
        },
        "groups": {name: audit_group(name, p) for name, p in data.items()},
    }
    for name, g in out["groups"].items():
        print(f"{name}: 碎片 {g['n_pieces']} -> 母介质 {g['n_media']} 根 "
              f"({g['pieces_per_medium']:.3f} 碎片/根), 体积分数 {g['volume_fraction']:.4%}")
        print(f"    周期盒 {g['periodic_box_nm']}; 最短碎片 {g['piece_length_min_nm']:.0f} nm; "
              f"    丢弃 {g['dropped_length_nm']:.0f} nm ({g['n_media_incomplete']} 根不完整, "
              f"    最长丢弃碎片 {g['dropped_fragment_len_max_nm']:.0f} nm) ")
        print(f"    E|u| = ({g['mean_abs_u'][0]:.3f},{g['mean_abs_u'][1]:.3f},{g['mean_abs_u'][2]:.3f}); "
              f"    KS 各向同性 p={g['ks_isotropic']['p']:.2e}, "
              f"    KS 极角均匀 p={g['ks_polar_uniform']['p']:.3f}")

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "data_audit.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/validate.py

```

#!/usr/bin/env python3
""" 几何内核的独立校验。

```

论文里所有结论都建立在两个原语上：线段最短距离和边界截断。这两个错了，后面所有概率都是错的，而且错得很隐蔽。所以在跑任何仿真之前先把它们钉死：

T1 线段最短距离 *vs* 暴力采样；
T2 题面图 2 给出的截断算例，逐坐标对照；

T3 用附件真实数据做往返测试：把碎片还原成母介质轴线，再按本模块的规则重新截断，
看能不能一模一样地还原出附件里的碎片（这同时校验了附件的周期盒尺寸判断）；
T4 球柱体近似相对平端面圆柱的误差量级；
T5 并查集导通判定在人工构造的通/断算例上的行为。

退出码 0 表示全部通过。

"""

```
from __future__ import annotations
```

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from load_attachment import GROUP_BOX, group_by_medium, load_pieces, reconstruct_axis
```

```
from microstructure import (EDGE, GAP, R_A, DSU, percolates, seg_seg_dist,
                             wrap_segment)
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
report: dict[str, object] = {}
```

```
failures: list[str] = []
```

```
def check(name: str, ok: bool, detail: object) -> None:
```

```
    report[name] = {"pass": bool(ok), "detail": detail}
```

```
    print(f"[{'PASS' if ok else 'FAIL'}] {name}: {detail}")
```

```
    if not ok:
```

```
        failures.append(name)
```

```
# ----- T1 线段距离
```

```
def t1_segment_distance() -> None:
```

```
    rng = np.random.default_rng(20260810)
```

```
    worst = 0.0
```

```
    for _ in range(300):
```

```
        p1, q1, p2, q2 = (rng.uniform(-1000, 1000, 3) for _ in range(4))
```

```
        exact = float(seg_seg_dist(p1[None, None], q1[None, None],
```

```
                                   p2[None, None], q2[None, None])[0, 0])
```

```
        t = np.linspace(0, 1, 900)[: , None]
```

```
        a = p1 + t * (q1 - p1)
```

```
        b = p2 + t * (q2 - p2)
```

```
        brute = float(np.min(np.linalg.norm(a[:, None, :] - b[None, :, :], axis=-1)))
```

```
        # 采样只能给出上界，精确解不应超过它，且不应显著小于它
```

```
        worst = max(worst, exact - brute)
```

```
    check("T1_ 线段最短距离", worst <= 1e-9,
```

```
          f" 解析解相对 900×900 暴力采样的最大超出量 = {worst:.3e} nm (应 0) ")
```

```
# ----- T2 题面算例
```

```
def t2_statement_example() -> None:
```

```
    box = np.full(3, EDGE)
```

```
    p = np.array([3500.0, 100.0, 200.0])
```

```
    q = np.array([6000.0, 150.0, 250.0])
```

```
    segs = wrap_segment(p, q, box)
```

```

xs = sorted(round(float(v), 6) for s in segs for v in (s[0][0], s[1][0]))
ok = (len(segs) == 2 and abs(xs[0] + 5000) < 1e-6 and abs(xs[-1] - 5000) < 1e-6)
# 题面: 超出的 X1 段 (5000..6000) 平移成 (-5000..-4000)
tail = [s for s in segs if s[0][0] < -4000][0]
ok = ok and abs(tail[0][0] + 5000) < 1e-6 and abs(tail[1][0] + 4000) < 1e-6
check("T2_ 题面截断算例", ok,
      f" 切成 {len(segs)} 段, 越界段 X 由 (5000,6000) 平移为 "
      f"({tail[0][0]:.1f},{tail[1][0]:.1f}), 与题面图 2 一致")

# ----- T3 附件往返
def t3_attachment_roundtrip() -> None:
    data = load_pieces()
    detail = {}
    totals = {"matched": 0, "complete": 0}
    all_ok = True
    for name, pieces in data.items():
        box = GROUP_BOX[name]
        n_complete = n_match = 0
        for idx in group_by_medium(pieces):
            start, end, total = reconstruct_axis(pieces, idx, box)
            if abs(total - 5000.0) > 1e-3:
                continue # 附件丢了短碎片的介质, 跳过
            n_complete += 1
            got = wrap_segment(start, end, box)
            want = [(pieces[i][:3], pieces[i][3:]) for i in idx]
            if len(got) != len(want):
                continue
            key = lambda s: tuple(np.round(np.concatenate(s), 4))
            if sorted(map(key, got)) == sorted(map(key, want)):
                n_match += 1
            ok = n_complete > 0 and n_match == n_complete
            all_ok &= ok
            detail[name] = f"{n_match}/{n_complete} 根完整介质的截断结果与附件逐坐标一致"
            totals["matched"] += n_match
            totals["complete"] += n_complete
        detail["合计"] = f"{totals['matched']}/{totals['complete']}"
        detail["matched_total"] = totals["matched"]
        detail["complete_total"] = totals["complete"]
    check("T3_ 附件截断往返", all_ok, detail)

# ----- T4 球柱近似
def t4_capsule_approximation() -> None:
    """ 平端面圆柱与球柱体只在端帽处不同, 估计它由此多判/少判接触的概率量级。"""
    rng = np.random.default_rng(7)
    n = 200000
    # 在 " 轴线距离刚好落在判定阈值附近 " 的壳层里采样, 看端帽区域占多大比例
    L, r = 5000.0, R_A
    c1 = np.zeros((n, 3))
    u1 = np.array([1.0, 0.0, 0.0])
    u2 = rng.normal(size=(n, 3)); u2 /= np.linalg.norm(u2, axis=1, keepdims=True)
    c2 = rng.uniform(-L / 2 - r, L / 2 + r, size=(n, 3))
    c2[:, 1:] = rng.uniform(-2 * r - GAP, 2 * r + GAP, size=(n, 2))
    p1 = c1 - L / 2 * u1; q1 = c1 + L / 2 * u1
    p2 = c2 - L / 2 * u2; q2 = c2 + L / 2 * u2

```

```

d = seg_seg_dist(p1[:, None], q1[:, None], p2[:, None], q2[:, None])[:, 0]
near = np.abs(d - (2 * r + GAP)) < GAP
# 端帽影响只出现在最近点落在轴线端点  $\pm r$  范围内的情形
frac_end = float(np.mean(near & (np.abs(c2[:, 0]) > L / 2 - r)))
frac_near = float(np.mean(near))
ratio = frac_end / max(frac_near, 1e-12)
check("T4_ 球柱体近似", ratio < 0.05,
      f" 临界接触中受端帽形状影响的比例 {ratio:.2%} (体积差  $4r/3h = \{4*r/(3*5000):.2%\}$  ) ")

# ----- T5 导通判定
def t5_percolation_logic() -> None:
    half = EDGE / 2
    # (a) 一根横贯左右的棒: 必导通
    p = np.array([[half + 1.0, 0.0, 0.0]]); q = np.array([[half - 1.0, 0.0, 0.0]])
    a = percolates(p, q)
    # (b) 两根中间留 100 nm 空隙: 不导通
    p2 = np.array([[half, 0, 0], [50.0, 0, 0]])
    q2 = np.array([[half, 0, 0], [50.0, 0, 0]])
    b = not percolates(p2, q2)
    # (c) 同样两根, 空隙缩到 1.0 nm(< GAP): 导通
    p3 = np.array([[half, 0, 0], [0.5, 0, 0]])
    q3 = np.array([[half, 0, 0], [0.5, 0, 0]])
    c = percolates(p3, q3)
    # (d) 只碰左面: 不导通
    d = not percolates(np.array([[half, 0, 0]]), np.array([[0.0, 0, 0]]))
    # (e) 碎片粘合开关: 一根跨 X 边界的棒, 碎片独立时不通, 粘合时通
    box = np.full(3, EDGE)
    segs = wrap_segment(np.array([3000.0, 0, 0]), np.array([8000.0, 0, 0]), box)
    sp = np.array([s[0] for s in segs]); sq = np.array([s[1] for s in segs])
    own = np.zeros(len(segs), dtype=int)
    e = (not percolates(sp, sq)) and percolates(sp, sq, owner_rod=own, bond_fragments=True)
    check("T5_ 导通判定逻辑", all([a, b, c, d, e]),
          f" 贯通 ={a} 断开 ={b} 间隙 1nm 导通 ={c} 单边不通 ={d} 碎片粘合开关 ={e}")

# ----- T6 加速等价
def t6_blocked_equals_naive() -> None:
    """ 分块 + 包围盒预筛的接触检索必须与朴素两两比较逐位一致。

    加速实现如果漏掉一条边, 导通概率会系统性偏低, 而这种偏差在结果里看不出来——
    所以它必须被测, 而不是被相信。
    """
    from microstructure import contact_pairs, sample_rods
    rng = np.random.default_rng(31415)
    box = np.full(3, EDGE)
    ok = True
    detail = []
    for n in (50, 150, 400):
        sp, sq, _ = sample_rods(n, rng, box)
        d = seg_seg_dist(sp[:, None], sq[:, None], sp[None, :, :], sq[None, :, :])
        iu = np.triu_indices(len(sp), k=1)
        hit = d[iu] <= 2 * R_A + GAP
        naive = set(zip(iu[0][hit].tolist(), iu[1][hit].tolist()))
        fast = set(map(tuple, contact_pairs(sp, sq, 2 * R_A + GAP).tolist()))
        ok &= naive == fast

```

```

        detail.append(f"N_A={n}: {len(naive)} 条边, 一致={naive == fast}")
    check("T6_ 加速实现等价", ok, "; ".join(detail))

if __name__ == "__main__":
    t1_segment_distance()
    t2_statement_example()
    t3_attachment_roundtrip()
    t4_capsule_approximation()
    t5_percolation_logic()
    t6_blocked_equals_naive()
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "validation.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    print("\n" + (" 全部通过" if not failures else f" 失败项: {failures}")
    sys.exit(1 if failures else 0)

```

src/theory_check.py

```
#!/usr/bin/env python3
```

""" 把仿真结果和渗流理论的独立估计一对对。

蒙特卡洛能给出概率，但给不出“这个数合不合理”。经典的排除体积判据（Balberg 等，1984）给出随机取向细长杆的渗流阈值满足

$$n_c \cdot \langle V_{ex} \rangle = B_c \approx 1.4,$$

其中 $\langle V_{ex} \rangle$ 是两根杆按连通判据的平均排除体积。用它算一个独立的阈值估计，和仿真给出的跃变中点比较：两者量级一致，说明仿真没有系统性地错；两者的差异方向（仿真阈值更低）也应当能被解释——本题中是有限尺寸效应（杆长恰为盒边一半）和方向偏向带电极面法向共同造成的。

同时把问题二结果表里的跃变中点用线性插值算出来，避免论文里出现手推的数。

结果写入 `results/theory_check.json`。

"""

```
from __future__ import annotations
```

```
import json
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from microstructure import GAP, H_A, R_A, V_A, V_BOX
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
B_C = 1.4 # 细长杆极限下的临界总排除体积
```

```
def excluded_volume_threshold() -> dict:
```

""" 按排除体积判据估计只填介质 A 时的渗流阈值体积分数。"""

```
L = H_A
```

```
# 连通判据等价于把杆加粗到"轴间距"  $D'$ ,  $D' = 2r +$ 
```

```
d_eff = 2 * R_A + GAP
```

```
# 两个随机取向球柱体的平均排除体积 ( $\langle \sin^2 \theta \rangle = 1/4$ )
```

```
v_ex = (np.pi / 2) * L ** 2 * d_eff + 2 * np.pi * L * d_eff ** 2 + (4 * np.pi / 3) * d_eff ** 3
```

```

n_c = B_C / v_ex # 临界数密度 (1/nm^3)
n_rods = n_c * V_BOX
return {
    "B_c": B_C,
    "d_eff_nm": d_eff,
    "mean_excluded_volume_nm3": float(v_ex),
    "critical_number_density_per_nm3": float(n_c),
    "critical_n_rods": float(n_rods),
    "critical_volume_fraction": float(n_rods * V_A / V_BOX),
}

def midpoint_from_simulation() -> dict | None:
    """ 从问题二的结果表线性插值出  $P=0.5$  的体积分数。 """
    p = RESULTS / "p2_probabilities.json"
    if not p.exists():
        return None
    data = json.loads(p.read_text(encoding="utf-8"))
    out = {}
    for mode, rows in data["runs"].items():
        xs = [r["phi_a"] for r in rows]
        ys = [r["p"] for r in rows]
        mid = None
        for (x0, y0), (x1, y1) in zip(zip(xs, ys), zip(xs[1:], ys[1:])):
            if y0 <= 0.5 <= y1:
                mid = x0 + (x1 - x0) * (0.5 - y0) / (y1 - y0)
                break
        out[mode] = {"phi_at_P50": mid}
    return out

def mode_contrast() -> dict | None:
    """ 两套方向口径在各档体积分数上的概率差，以及主口径的逐档增量。

    论文正文引用了这些差值；在这里算出来，它们才有出处，而不是正文里手推的数。
    """
    p = RESULTS / "p2_probabilities.json"
    if not p.exists():
        return None
    d = json.loads(p.read_text(encoding="utf-8"))
    pol = {r["phi_nominal"]: r["p"] for r in d["runs"]["polar_uniform"]}
    iso = {r["phi_nominal"]: r["p"] for r in d["runs"]["isotropic"]}
    phis = sorted(pol)
    return {
        "gap_polar_minus_isotropic": {f"{k:.2%}": pol[k] - iso[k] for k in phis},
        "increment_polar": {f"{a:.2%}->{b:.2%}": pol[b] - pol[a]
                            for a, b in zip(phis[:-1], phis[1:])},
    }

def marginal_efficiency() -> dict | None:
    """ 从问题四的网格算两种介质的 " 每元换来多少导通概率 "。

    这是问题四结论的机理：介质  $B$  单价便宜，但在必须达到  $P 0.90$  的那一段
    ( $N_A$  接近渗流阈值)，介质  $A$  的边际收益陡得多。两者的效率在  $N_A 400$ 
    附近交叉，而约束把可行解逼到交叉点右侧，于是最优解退化为纯  $A$ 。
    """

```

```

"""
p = RESULTS / "p4_cost_optimum.json"
if not p.exists():
    return None
d = json.loads(p.read_text(encoding="utf-8"))
c_a, c_b = d["cost_per_rod_yuan"], d["cost_per_sphere_yuan"]
grid = {(g["n_a"], g["n_b"]): g["p"] for g in d["grid"]}
nas = sorted({k[0] for k in grid})
nbs = sorted({k[1] for k in grid})
out = {}
for i, na in enumerate(nas):
    row = {}
    if i > 0:
        # dP/dN_A (沿 N_B=0)
        prev = nas[i - 1]
        dp = grid[(na, 0)] - grid[(prev, 0)]
        row["dP_dNA_per_yuan"] = (dp / (na - prev)) / c_a
    if len(nbs) > 1:
        # dP/dN_B (在 N_B=0 处)
        nb1 = nbs[1]
        dp = grid[(na, nb1)] - grid[(na, 0)]
        row["dP_dNB_per_yuan"] = (dp / nb1) / c_b
    if row:
        row["P_at_NB0"] = grid[(na, 0)]
        out[str(na)] = row
return out

def main() -> int:
    ev = excluded_volume_threshold()
    mid = midpoint_from_simulation()
    out = {"excluded_volume_estimate": ev, "simulation_midpoint": mid,
          "mode_contrast": mode_contrast(),
          "marginal_efficiency": marginal_efficiency()}
    print(" 排除体积判据 (独立于仿真的理论估计): ")
    print(f" 平均排除体积 <V_ex> = {ev['mean_excluded_volume_nm3']:.4e} nm^3")
    print(f" 临界根数 N_c   {ev['critical_n_rods']:.0f}, "
          f" 对应体积分数 {ev['critical_volume_fraction']:.4%}")
    if mid:
        for m, v in mid.items():
            if v["phi_at_P50"]:
                print(f" 仿真跃变中点 ({m}): (P=0.5)   {v['phi_at_P50']:.4%}")
    if out["mode_contrast"]:
        print(" 两套方向口径的概率差: ",
              {k: round(v, 4) for k, v in out["mode_contrast"]["gap_polar_minus_isotropic"].items()})
        print(" 主口径逐档增量: ",
              {k: round(v, 4) for k, v in out["mode_contrast"]["increment_polar"].items()})
    if out["marginal_efficiency"]:
        print(" 每元边际收益 (ΔP/元): ")
        for na, v in out["marginal_efficiency"].items():
            a = v.get("dP_dNA_per_yuan"); b = v.get("dP_dNB_per_yuan")
            print(f"   N_A={na:>4}   P(N_B=0)={v['P_at_NB0']:.3f}   "
                  f"   介质 A={'—' if a is None else f'{a:.3f}'}   "
                  f"   介质 B={'—' if b is None else f'{b:.3f}'}")
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "theory_check.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

src/cluster_stats.py

```
#!/usr/bin/env python3
```

```
""" 渗流的结构证据：最大团簇占比随体积分数的变化。
```

导通概率 P 只告诉我们“通没通”，看不出通的时候内部长什么样。渗流理论里刻画这件事的量是 **序参量**——最大连通团簇所含碎片占全部碎片的比例 $S_{\max}$ 。若体系真的经历渗流相变， $S_{\max}$ 会在与 P_c 跃变同一位置由接近 0 迅速升到 $O(1)$ ；若只是有限尺寸下的偶然贯通，两者的位置就不会对上。

这是一条独立于导通判定的检查： $S_{\max}$ 的计算完全不涉及带电面。

```
结果写入 results/cluster_stats.json。
"""
```

```
from __future__ import annotations
```

```
import json
from collections import Counter
from pathlib import Path
```

```
import numpy as np
```

```
from microstructure import (EDGE, GAP, R_A, DSU, contact_pairs, percolates,
                             sample_rods)
from simulate import n_rods_for_phi
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
BOX = np.full(3, EDGE)
FRACTIONS = [0.0030, 0.0040, 0.0050, 0.0060, 0.0070, 0.0080, 0.0090, 0.0100]
TRIALS = 200
```

```
def largest_cluster_fraction(sp: np.ndarray, sq: np.ndarray) -> tuple[float, int]:
```

```
    """ 返回 (最大团簇碎片占比, 团簇数)。只看介质之间的连通性，不涉及带电面。"""
```

```
    n = len(sp)
    if n == 0:
        return 0.0, 0
    dsu = DSU(n)
    for a, b in contact_pairs(sp, sq, 2 * R_A + GAP):
        dsu.union(int(a), int(b))
    sizes = Counter(dsu.find(i) for i in range(n))
    return max(sizes.values()) / n, len(sizes)
```

```
def main() -> int:
```

```
    rng = np.random.default_rng(20260810)
    rows = []
    for phi in FRACTIONS:
        n_a = n_rods_for_phi(phi)
        smax, ncl, hits = [], [], 0
```

```

for _ in range(TRIALS):
    sp, sq, _ = sample_rods(n_a, rng, BOX, orientation="polar_uniform")
    f, k = largest_cluster_fraction(sp, sq)
    smax.append(f)
    ncl.append(k)
    hits += bool(percolates(sp, sq))
row = {
    "phi": phi, "n_a": n_a, "trials": TRIALS,
    "s_max_mean": float(np.mean(smax)), "s_max_sd": float(np.std(smax, ddof=1)),
    "n_clusters_mean": float(np.mean(ncl)),
    "p_percolate": hits / TRIALS,
}
rows.append(row)
print(f"    ={{phi:.2%}} N_A={{n_a:4d}} S_max={{row['s_max_mean']:.3f}}"
      f"±{{row['s_max_sd']:.3f}} 团簇数 ={{row['n_clusters_mean']:.1f}}"
      f" P={{row['p_percolate']:.3f}}")

RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "cluster_stats.json").write_text(
    json.dumps({"trials": TRIALS, "orientation": "polar_uniform", "rows": rows},
               ensure_ascii=False, indent=2), encoding="utf-8")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/solve_p1.py

```
#!/usr/bin/env python3
```

```
""" 问题一：判定附件给出的三个微构体是否导通。
```

附件给的是 **** 碎片 **** 而不是母介质，并且系统性地丢弃了长度小于约 *500 nm* 的短碎片（见 *audit_attachment.py*）。丢掉的碎片有可能恰好是一座桥，所以本脚本对每一组都在三种口径下判一次，只有三种口径给出同一答案时，结论才算稳：

as_given —— 完全按附件的碎片判（这是题目直接给的数据）；
restored —— 先把母介质的轴线还原成完整的 *5000 nm*，再重新截断，补回被丢弃的短碎片；
bonded —— 在 *restored* 的基础上，把同一根母介质的碎片视为电学一体（对照口径）。

```
结果写入 results/p1_connectivity.json。
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from load_attachment import GROUP_BOX, group_by_medium, load_pieces
```

```
from microstructure import EDGE, GAP, R_A, percolates, wrap_segment
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```

def reconstruct_full_axis(pieces: np.ndarray, idx: list[int], box: np.ndarray) -> tuple[np.ndarray,
↳ np.ndarray]:
    """ 把一根母介质的所有碎片摊回未截断的直线上, 返回完整 5000 nm 轴线段。

    做法: 以第一个碎片的起点为原点、其方向为  $u$ , 对每个碎片搜索使其回到同一条直线上的
    整数周期偏移  $k$  ( $|k| \leq 6$  足够, 因为轴长 5000、最小盒宽 1000)。得到各碎片在该直线上的
    参数区间后, 缺口即被丢弃的短碎片; 若缺口在两端, 按端点是否贴边界决定补在哪一侧。
    """
    o = pieces[idx[0]][:3].astype(float)
    d = pieces[idx[0]][3:] - pieces[idx[0]][:3]
    u = d / np.linalg.norm(d)

    ks = np.array(np.meshgrid(*[np.arange(-6, 7)] * 3, indexing="ij")).reshape(3, -1).T
    offsets = ks * box

    intervals: list[tuple[float, float]] = []
    for i in idx:
        best = None
        for pt_a, pt_b in [(pieces[i][:3], pieces[i][3:]):
            cand = pt_a + offsets - o
            perp = cand - np.outer(cand @ u, u)
            j = int(np.argmin(np.linalg.norm(perp, axis=1)))
            err = float(np.linalg.norm(perp[j]))
            shift = offsets[j]
            ta = float((pt_a + shift - o) @ u)
            tb = float((pt_b + shift - o) @ u)
            best = (min(ta, tb), max(ta, tb), err)
        intervals.append((best[0], best[1]))
    intervals.sort()

    covered = sum(b - a for a, b in intervals)
    t_lo, t_hi = intervals[0][0], intervals[-1][1]
    interior = (t_hi - t_lo) - covered
    remaining = 5000.0 - covered - interior
    if remaining > 1e-6:
        half = box / 2.0
        head_pt = o + t_lo * u
        head_on_edge = any(abs(abs(head_pt[j]) - half[j]) < 1e-6 for j in range(3))
        tail_pt = o + t_hi * u
        tail_on_edge = any(abs(abs(tail_pt[j]) - half[j]) < 1e-6 for j in range(3))
        if head_on_edge and not tail_on_edge:
            t_lo -= remaining
        elif tail_on_edge and not head_on_edge:
            t_hi += remaining
        else:
            t_lo -= remaining / 2.0
            t_hi += remaining / 2.0
    return o + t_lo * u, o + t_hi * u

def pieces_to_arrays(pieces: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    media = group_by_medium(pieces)
    owner = np.empty(len(pieces), dtype=int)
    for m, idx in enumerate(media):
        for i in idx:
            owner[i] = m

```

```

return pieces[:, :3].copy(), pieces[:, 3:].copy(), owner

def restored_arrays(pieces: np.ndarray, box: np.ndarray):
    sp, sq, own = [], [], []
    for m, idx in enumerate(group_by_medium(pieces)):
        a, b = reconstruct_full_axis(pieces, idx, box)
        for s, e in wrap_segment(a, b, box):
            sp.append(s); sq.append(e); own.append(m)
    return np.array(sp), np.array(sq), np.array(own, dtype=int)

def contact_stats(sp, sq):
    """ 接触图的规模，用于论文里说明结论不是靠一两条边撑着的。 """
    from microstructure import seg_seg_dist
    d = seg_seg_dist(sp[:, None, :], sq[:, None, :], sp[None, :, :], sq[None, :, :])
    iu = np.triu_indices(len(sp), k=1)
    n_edge = int(np.sum(d[iu] <= 2 * R_A + GAP))
    half = EDGE / 2
    lo = np.minimum(sp[:, 0], sq[:, 0]) - R_A
    hi = np.maximum(sp[:, 0], sq[:, 0]) + R_A
    return {
        "n_fragments": int(len(sp)),
        "n_contact_edges": n_edge,
        "n_touch_left": int(np.sum(lo <= -half + GAP)),
        "n_touch_right": int(np.sum(hi >= half - GAP)),
        "min_pair_surface_gap_nm": float(np.min(d[iu]) - 2 * R_A),
    }

def main() -> int:
    data = load_pieces()
    out = {}
    for name, pieces in data.items():
        box = GROUP_BOX[name]
        sp0, sq0, own0 = pieces_to_arrays(pieces)
        sp1, sq1, own1 = restored_arrays(pieces, box)
        res = {
            "n_pieces_in_attachment": int(len(pieces)),
            "n_media": int(own0.max() + 1),
            "n_fragments_restored": int(len(sp1)),
            "box_nm": box.tolist(),
            "as_given": bool(percolates(sp0, sq0)),
            "restored": bool(percolates(sp1, sq1)),
            "bonded": bool(percolates(sp1, sq1, owner_rod=own1, bond_fragments=True)),
            "graph_as_given": contact_stats(sp0, sq0),
        }
    # 判据阈值扰动： 是题目给死的，扫一遍是为了说明结论不是卡在阈值上
    import microstructure as ms
    gap_scan = {}
    old = ms.GAP
    try:
        for g in (0.9, 1.35, 1.8, 2.25, 2.7):
            ms.GAP = g
            gap_scan[f"{g:g}"] = bool(percolates(sp0, sq0))
    finally:
        ms.GAP = old

```

```

        ms.GAP = old
    res["gap_scan"] = gap_scan
    res["gap_robust"] = len(set(gap_scan.values())) == 1

    res["conclusion"] = " 导通" if res["as_given"] else " 不导通"
    res["robust"] = bool(res["as_given"] == res["restored"])
    out[name] = res
    print(f"{name}: 母介质 {res['n_media']} 根 / 碎片 {res['n_pieces_in_attachment']} 个 -> "
          f"{res['conclusion']} (as_given={res['as_given']}, restored={res['restored']}, "
          f"bonded={res['bonded']})")
    print(f"      接触图: {res['graph_as_given']}")

RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "p1_connectivity.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/solve_p2.py

```

#!/usr/bin/env python3
""" 问题二: 只填介质 A 时, 体积分数 0.50%/0.60%/0.70%/1.00% 的导通概率。

对每个体积分数做 T 次独立仿真, 导通频率即概率估计, 附 Wilson 95% 置信区间。
主口径用附件标定的方向分布 (polar_uniform), 同时给出球面均匀 (isotropic) 的对照——
这两个分布给出的概率差别可达 20 个百分点, 是全题最敏感的建模选择。
"""

from __future__ import annotations

import json
import time
from pathlib import Path

from simulate import Config, estimate_p, n_rods_for_phi

RESULTS = Path(__file__).resolve().parents[1] / "results"
FRACTIONS = [0.0050, 0.0060, 0.0070, 0.0100]
TRIALS = 8000

def main() -> int:
    out = {"trials_per_point": TRIALS, "fractions": FRACTIONS, "runs": {}}
    for mode in ("polar_uniform", "isotropic"):
        rows = []
        for phi in FRACTIONS:
            n = n_rods_for_phi(phi)
            t0 = time.time()
            r = estimate_p(Config(n_a=n, orientation=mode), TRIALS)
            r["phi_nominal"] = phi
            r["seconds"] = round(time.time() - t0, 1)
            rows.append(r)
        print(f"{mode:14s} = {phi:.2%} N_A={n:4d} P={r['p']:.4f} ")

```

```

        f"{r['ci_lo']:.4f},{r['ci_hi']:.4f]} ({r['seconds']}s)")
    out["runs"][mode] = rows

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "p2_probabilities.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/solve_p3.py

```
#!/usr/bin/env python3
```

```
""" 问题三：只填介质 A 时，使导通概率不低于 90% 的最低体积分数。
```

两步：

1. 粗扫 + 二项似然下的 *logit* 拟合，把 $P=0.90$ 的穿越点定位到几根棒以内；
2. 在 0.01% (题目要求的精度) 的体积分数网格上逐点大样本复算，取 ** 最小的 **、点估计与 *Wilson* 下界都不低于 0.90 的那一格作为答案。

只用拟合曲线反解会把答案的可信度压在模型形式上；第 2 步是直接的频率验证，拟合只用来省算力。

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import time
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from scipy.optimize import minimize
```

```
from microstructure import V_A, V_BOX
```

```
from simulate import Config, estimate_p, n_rods_for_phi
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
TARGET = 0.90
```

```
# 样本量按分辨率需要定：相邻 0.01% 网格点相差约 7 根棒，对应  $\Delta P$  0.02；
```

```
#  $T=4000$  时 Wilson 半宽约 0.009，足以把相邻网格点分开。
```

```
COARSE_TRIALS = 2000
```

```
FINE_TRIALS = 4000
```

```
FINAL_TRIALS = 12000
```

```
def logit_fit(ns: np.ndarray, hits: np.ndarray, trials: np.ndarray) -> tuple[float, float]:
```

```
    """ 二项似然拟合 logit  $P = a + b \cdot N$ ，返回  $(a, b)$ 。 """
```

```
    def nll(theta):
```

```
        a, b = theta
```

```
        z = a + b * ns
```

```
        # log-sum-exp 稳定写法
```

```
        ll = hits * z - trials * np.logaddexp(0.0, z)
```

```
        return -np.sum(ll)
```

```
    x0 = np.array([-8.0, 0.015])
```

```

res = minimize(nll, x0, method="Nelder-Mead",
               options={"xatol": 1e-8, "fatol": 1e-8, "maxiter": 20000})
return float(res.x[0]), float(res.x[1])

def solve_for_mode(mode: str) -> dict:
    print(f"\n=== 方向分布: {mode} ===")
    coarse_ns = [n_rods_for_phi(p) for p in (0.0060, 0.0068, 0.0076, 0.0084, 0.0092, 0.0100)]
    rows = []
    for n in coarse_ns:
        r = estimate_p(Config(n_a=n, orientation=mode), COARSE_TRIALS)
        rows.append(r)
        print(f" 粗扫 N_A={n:4d}  P={r['p']:.4f}")

    ns = np.array([r["n_a"] for r in rows], float)
    hits = np.array([r["hits"] for r in rows], float)
    tri = np.array([r["trials"] for r in rows], float)
    a, b = logit_fit(ns, hits, tri)
    n_star = (np.log(TARGET / (1 - TARGET)) - a) / b
    phi_star = n_star * V_A / V_BOX
    print(f"  logit 拟合: P=0.90 处 N_A {n_star:.1f}  {phi_star:.4f}")

    # 在 0.01% 网格上逐点复算
    center = round(phi_star * 10000) / 10000 # 归到 0.01% 网格
    grid = [round(center + k * 0.0001, 6) for k in (-2, -1, 0, 1, 2)]
    fine = []
    for phi in grid:
        n = n_rods_for_phi(phi)
        t0 = time.time()
        r = estimate_p(Config(n_a=n, orientation=mode), FINE_TRIALS)
        r["phi_grid"] = phi
        fine.append(r)
        print(f"  细算  P={phi:.2%} N_A={n:4d}  P={r['p']:.4f} "
              f"[{r['ci_lo']:.4f},{r['ci_hi']:.4f}]  ({time.time()-t0:.0f}s)")

    ok = [r for r in fine if r["p"] >= TARGET]
    answer = min(ok, key=lambda r: r["phi_grid"]) if ok else None

    final = None
    if answer is not None:
        print(f"  复核  P={answer['phi_grid']:.2%} 用 {FINAL_TRIALS} 次试验 ...")
        final = estimate_p(Config(n_a=answer["n_a"], orientation=mode), FINAL_TRIALS,
                           seed=987654321)
        final["phi_grid"] = answer["phi_grid"]
        print(f"  复核结果 P={final['p']:.4f} [{final['ci_lo']:.4f},{final['ci_hi']:.4f}]")

    return {
        "mode": mode, "coarse": rows, "logit_a": a, "logit_b": b,
        "n_star_fit": n_star, "phi_star_fit": phi_star,
        "fine_grid": fine,
        "answer_phi": answer["phi_grid"] if answer else None,
        "answer_n_a": answer["n_a"] if answer else None,
        "final_check": final,
    }
}

```

```

def main() -> int:
    out = {"target": TARGET,
          "trials": {"coarse": COARSE_TRIALS, "fine": FINE_TRIALS, "final": FINAL_TRIALS},
          "modes": {}}
    for mode in ("polar_uniform", "isotropic"):
        out["modes"][mode] = solve_for_mode(mode)
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "p3_threshold.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    for m, r in out["modes"].items():
        print(f"\n{m}: 最低体积分数 = {r['answer_phi']:.2%} (N_A={r['answer_n_a']}) ")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/solve_p4.py

```
#!/usr/bin/env python3
```

""" 问题四：同时填 A 、 B 两种介质，在导通概率不低于 90% 的前提下最小化总成本。

成本：介质 A 1.05 元/ $m^3 \rightarrow$ 每根 0.0148441 元；介质 B 0.05 元/ $m^3 \rightarrow$ 每颗 0.00167552 元。
一颗 B 的价钱只有一根 A 的 $1/8.9$ ，但 B 是半径 200 nm 的球，跨越能力远不如长 5000 nm 的棒，
所以两者之间存在真实的权衡，而不是“谁便宜用谁”。

求解结构

决策变量 (N_A, N_B) 都是整数，目标线性，可行域由 $P(N_A, N_B)$ 0.90 给出。
 P 对 N_A, N_B 都单调不减（加介质只会增加接触图的点和边，不会删掉任何通路），
因此最优解一定落在可行域边界 $N_B = g(N_A)$ 上，问题退化为一维搜索。

直接对每个 N_A 二分求 $g(N_A)$ 要跑几百万次仿真。这里改成两段：

阶段 A ：在 (N_A, N_B) 网格上用中等样本量估 P ，拟合 *logit* 代理模型；

阶段 B ：用代理模型定位边界与最优点，再对最优点及其邻域做大样本直接验证。

代理模型只用来省算力，最终结论以阶段 B 的直接频率为准。

"""

```
from __future__ import annotations
```

```
import itertools
```

```
import json
```

```
import time
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from scipy.optimize import minimize
```

```
from microstructure import COST_A, COST_B, V_A, V_B, V_BOX
```

```
from simulate import Config, estimate_p
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
TARGET = 0.90
```

```
GRID_A = [150, 250, 320, 380, 440, 500, 560]
```

```
GRID_B = [0, 300, 700, 1200, 2000, 3200]
```

```

GRID_TRIALS = 700
VERIFY_TRIALS = 6000

def design_matrix(na: np.ndarray, nb: np.ndarray) -> np.ndarray:
    """ 代理模型的特征。

    渗流概率对根数近似呈 logit-线性, B 的边际贡献随 A 增多而变化 (协同项),
    再加一个 sqrt(N_B) 项刻画球的边际收益递减。
    """
    na = np.asarray(na, float)
    nb = np.asarray(nb, float)
    return np.column_stack([
        np.ones_like(na), na, nb, np.sqrt(nb), na * nb / 1000.0,
    ])

def fit_surrogate(na, nb, hits, trials):
    X = design_matrix(na, nb)
    y = np.asarray(hits, float)
    n = np.asarray(trials, float)

    def nll(w):
        z = np.clip(X @ w, -40, 40)
        return -np.sum(y * z - n * np.logaddexp(0.0, z)) + 1e-6 * np.sum(w * w)

    best = None
    for x0 in (np.zeros(X.shape[1]), np.array([-8.0, 0.02, 0.001, 0.0, 0.0])):
        r = minimize(nll, x0, method="Nelder-Mead",
                     options={"maxiter": 60000, "fatol": 1e-9, "xatol": 1e-9})
        if best is None or r.fun < best.fun:
            best = r
    return best.x

def p_hat(w, na, nb):
    z = design_matrix(np.atleast_1d(na), np.atleast_1d(nb)) @ w
    return 1.0 / (1.0 + np.exp(-z))

def min_nb_for(w, na, nb_max=8000):
    """ 代理模型下满足 P TARGET 的最小 N_B (不可行则返回 None)。 """
    if p_hat(w, na, 0)[0] >= TARGET:
        return 0
    if p_hat(w, na, nb_max)[0] < TARGET:
        return None
    lo, hi = 0, nb_max
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if p_hat(w, na, mid)[0] >= TARGET:
            hi = mid
        else:
            lo = mid
    return hi

```

```

def main() -> int:
    mode = "polar_uniform"
    print("=== 阶段 A: 网格取样, 拟合代理模型 ===")
    grid = []
    for na, nb in itertools.product(GRID_A, GRID_B):
        t0 = time.time()
        r = estimate_p(Config(n_a=na, n_b=nb, orientation=mode), GRID_TRIALS)
        grid.append(r)
        print(f"   N_A={na:4d} N_B={nb:5d} _A={r['phi_a']:.3%} _B={r['phi_b']:.2%} "
              f"P={r['p']:.3f} 成本={r['cost_yuan']:.3f} 元 ({time.time()-t0:.0f}s)")

    na = np.array([g["n_a"] for g in grid], float)
    nb = np.array([g["n_b"] for g in grid], float)
    hits = np.array([g["hits"] for g in grid], float)
    tri = np.array([g["trials"] for g in grid], float)
    w = fit_surrogate(na, nb, hits, tri)
    pred = p_hat(w, na, nb)
    resid = np.max(np.abs(pred - hits / tri))
    print(f"   代理模型最大绝对残差 = {resid:.3f}")

    print("\n=== 阶段 B: 沿可行边界一维搜索 ===")
    cand = []
    for a in range(120, 620, 4):
        b = min_nb_for(w, a)
        if b is None:
            continue
        cand.append({"n_a": a, "n_b": b, "cost": a * COST_A + b * COST_B})
    cand.sort(key=lambda c: c["cost"])
    # 代理模型的最优点附近成本几乎相同 (N_A 相差 4 根的点成本差不到 0.1%),
    # 全部拿去验证等于把算力花在同一个点上。按 N_A 至少相隔 40 根取互不重复的候选,
    # 既覆盖了边界的不同区段, 又能看出成本沿边界是否真的平坦。
    spread: list[dict] = []
    for c in cand:
        if all(abs(c["n_a"] - s["n_a"]) >= 40 for s in spread):
            spread.append(c)
        if len(spread) >= 5:
            break
    print("   代理模型给出的候选 (按成本排序, N_A 至少相隔 40): ")
    for c in spread:
        print(f"       N_A={c['n_a']:4d} N_B={c['n_b']:5d} 成本={c['cost']:.4f} 元")
    cand = spread

    print("\n=== 阶段 C: 候选点大样本直接验证 (含向上补 B 直到真正可行) ===")
    verified = []
    all_probes = [] # 未通过的探测点同样要留痕, 否则论文里的表无从溯源
    seen = set()
    for c in cand:
        a = c["n_a"]
        if a in seen:
            continue
        seen.add(a)
        b = c["n_b"]
        for _ in range(6):
            r = estimate_p(Config(n_a=a, n_b=b, orientation=mode), VERIFY_TRIALS)
            all_probes.append(r)
            print(f"       N_A={a:4d} N_B={b:5d} P={r['p']:.4f} ")

```

```

        f"{r['ci_lo']:.4f},{r['ci_hi']:.4f}] 成本={r['cost_yuan']:.4f} 元")
    if r["p"] >= TARGET and r["ci_lo"] >= TARGET - 0.01:
        verified.append(r)
        break
    b = int(b + max(60, 0.06 * max(b, 500)))
if not verified:
    print(" 没有候选点通过验证")
    return 1

best = min(verified, key=lambda r: r["cost_yuan"])
print(f"\n 最优: N_A={best['n_a']} N_B={best['n_b']} "
      f"_A={best['phi_a']:.3%} _B={best['phi_b']:.2%} "
      f"P={best['p']:.4f} 总成本={best['cost_yuan']:.4f} 元")

# 纯 A 参照: 直接取问题三的答案根数, 成本与混填最优做同口径比较
pure_a = None
p3 = RESULTS / "p3_threshold.json"
if p3.exists():
    n3 = json.loads(p3.read_text(encoding="utf-8"))["modes"][mode]["answer_n_a"]
    if n3:
        pure_a = estimate_p(Config(n_a=int(n3), n_b=0, orientation=mode), VERIFY_TRIALS)
        print(f" 纯 A 参照 N_A={n3} P={pure_a['p']:.4f} 成本={pure_a['cost_yuan']:.4f} 元")
out = {
    "target": TARGET, "mode": mode,
    "cost_per_rod_yuan": COST_A, "cost_per_sphere_yuan": COST_B,
    "grid_trials": GRID_TRIALS, "verify_trials": VERIFY_TRIALS,
    "grid": grid, "surrogate_weights": w.tolist(),
    "surrogate_max_abs_resid": float(resid),
    "candidates": cand[:12], "verified": verified, "all_probes": all_probes,
    "best": best,
    "pure_a_reference": pure_a,
}
RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "p4_cost_optimum.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_break_even.py

```
#!/usr/bin/env python3
```

""" 介质 B 的盈亏平衡价: 多便宜才值得掺? """

问题四的答案是“纯 A”, 但这是 ** 价格 ** 的结论而不是几何的结论。把它说清楚需要一个数:
介质 B 的单价降到多少, 最优解才会从纯 A 变成混填。

设可行边界为 $g(N_A)$ (即达到 $P \geq 0.90$ 所需的最小球数), 边界上的成本

$$C(N_A) = c_A N_A + c_B g(N_A), \quad dC/dN_A = c_A + c_B g'.$$

$g' < 0$ 。若 $c_A + c_B g' < 0$, 成本随 N_A 增大而下降, 最优在边界右端点 (纯 A);

反之应当少用 A、多用 B。盈亏平衡价即 $c_B^* = -c_A/g'$ 。

g' 直接由 `solve\_p4.py` 阶段 C 已验证通过 $**$ 的边界点线性拟合得到——
 这些点每个都用 6000 次试验确认了 $P \geq 0.90$, 比再拟合一层代理模型可靠。

结果写入 `results/p4_break_even.json`。

"""

from __future__ import annotations

import json

from pathlib import Path

import numpy as np

from microstructure import COST_A, COST_B, V_B

RESULTS = Path(__file__).resolve().parents[1] / "results"

def main() -> int:

src = RESULTS / "p4_cost_optimum.json"

if not src.exists():

print(" 先运行 solve_p4.py")

return 1

d = json.loads(src.read_text(encoding="utf-8"))

pts = sorted(((v["n_a"], v["n_b"]) for v in d["verified"]), key=lambda t: t[0])

边界只在 $g > 0$ 的区段上才有斜率可言。 N_B 已经取到 0 的点里, 只有 N_A 最小的那个

位于边界上 (再减 A 就必须补 B); 更大的 N_A 配 $N_B=0$ 是 "配足过头" 的内点,

把它们放进拟合会把斜率压平 ($-11.4 \rightarrow -9.1$), 进而把盈亏平衡价算高。

zeros = [t for t in pts if t[1] == 0]

if len(zeros) > 1:

keep = min(zeros, key=lambda t: t[0])

pts = [t for t in pts if t[1] > 0 or t == keep]

if len(pts) < 2:

print(" 已验证的边界点不足两个, 无法拟合斜率")

return 1

x = np.array([p[0] for p in pts], float)

y = np.array([p[1] for p in pts], float)

slope, intercept = np.polyfit(x, y, 1)

c_b_star = -COST_A / slope if slope < 0 else None

unit_price_now = COST_B / (V_B / 1e9) # 元/ m^3 , 应为 0.05

out = {

"boundary_points_verified": [{"n_a": int(a), "n_b": int(b)} for a, b in pts],

"slope_dNB_dNA": float(slope),

"intercept": float(intercept),

"cost_per_rod_yuan": COST_A,

"cost_per_sphere_yuan": COST_B,

"unit_price_now_yuan_per_um3": float(unit_price_now),

}

print(f" 已验证的边界点: {[int(a), int(b)] for a, b in pts}")

print(f" 边界斜率 g' = {slope:.4f} 颗/根 (少用 1 根 A 需补约 {-slope:.1f} 颗 B) ")

if c_b_star:

unit_star = c_b_star / (V_B / 1e9)

out["break_even_cost_per_sphere_yuan"] = float(c_b_star)

out["break_even_unit_price_yuan_per_um3"] = float(unit_star)

```

        out["required_price_cut_pct"] = float((1 - unit_star / unit_price_now) * 100)
        print(f" 盈亏平衡: 单颗 {c_b_star:.6e} 元, 即单价 {unit_star:.4f} 元/m³")
        print(f" 现价 {unit_price_now:.4f} 元/m³, 需再降 "
              f"{out['required_price_cut_pct']:.1f}% 才值得掺 B")

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "p4_break_even.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/sensitivity.py

```
#!/usr/bin/env python3
```

```
""" 灵敏度与稳健性分析。
```

要回答的不是“模型是否鲁棒”这种没法证伪的话，而是三个具体问题：

S1 判据阈值 $=1.8\text{ nm}$ 若变动 $\pm 50\%$ ，导通概率变多少？（是题目给死的，做这条是为了说明结论不是卡在阈值的某个巧合上）

S2 各条建模口径分别把导通概率推到哪里？逐条列出，包括被否掉的“碎片粘合”口径——它把 P 顶到 1.0 ，正是它被否掉的原因。

S3 蒙特卡洛样本量够不够？给出 P 随试验次数的收敛轨迹。

固定在 $=0.70\%$ （处于渗流跃变最陡的位置，对任何扰动都最敏感）。

```
"""
```

```

from __future__ import annotations

import json
from pathlib import Path

import numpy as np

import microstructure as ms
from simulate import Config, estimate_p, n_rods_for_phi, wilson

RESULTS = Path(__file__).resolve().parents[1] / "results"
PHI = 0.0070
TRIALS = 4000

def with_gap(gap: float, n_a: int, trials: int) -> dict:
    old = ms.GAP
    try:
        ms.GAP = gap
        r = estimate_p(Config(n_a=n_a), trials)
    finally:
        ms.GAP = old
    r["gap"] = gap
    return r

```

```

def main() -> int:
    n_a = n_rods_for_phi(PHI)
    out = {"phi": PHI, "n_a": n_a, "trials": TRIALS, "gap_scan": [], "assumption_table": []}

    print(f"={PHI:.2%}, N_A={n_a}")
    print("--- S1 判据阈值 ---")
    for gap in (0.9, 1.35, 1.8, 2.25, 2.7):
        # Linux 下 multiprocessing 默认 fork, 子进程继承已改写的 ms.GAP;
        # percolates 在调用时才查 microstructure 的模块全局, 因此改生效。
        # 样本量减半以控制时间。
        r = with_gap(gap, n_a, TRIALS // 2)
        out["gap_scan"].append(r)
        print(f"={gap:4.2f} nm P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}]")

    print("--- S2 建模口径 ---")
    variants = [
        (" 基准: 附件标定方向 + 碎片各自独立", Config(n_a=n_a)),
        (" 方向改为球面均匀", Config(n_a=n_a, orientation="isotropic")),
        (" 碎片粘合为同一导体 (已否决)", Config(n_a=n_a, bond_fragments=True)),
    ]
    for name, cfg in variants:
        r = estimate_p(cfg, TRIALS)
        r["name"] = name
        out["assumption_table"].append(r)
        print(f" {name}: P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}]")

    print("--- S3 样本量收敛 ---")
    conv = []
    for t in (250, 500, 1000, 2000, 4000, 8000):
        r = estimate_p(Config(n_a=n_a), t, seed=13579)
        conv.append({"trials": t, "p": r["p"], "ci_lo": r["ci_lo"], "ci_hi": r["ci_hi"],
                    "half_width": r["half_width"]})
        print(f" T={t:5d} P={r['p']:.4f} ±{r['half_width']:.4f}")
    out["convergence"] = conv

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "sensitivity.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2, encoding="utf-8")
    )
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/sphere_mode_check.py

```
#!/usr/bin/env python3
```

""" 介质 B 越界处理口径的对照 (假设 D8)。

球被边界切开后, 碎片其实是球缺。实现里按 " 整球置于其所在周期像 " 近似

(`sphere\_mode="wrap"`) , 在距壁 200 nm 的薄层内会略微高估它的跨越能力。

对照口径是把球心限制在 $[-(L/2-R), L/2-R]$, 保证整颗球都在盒内、不产生碎片

(`sphere\_mode="inside"`)。

只在含介质 B 的配置上比较才有意义; 问题四的最优解 $SN_B=0$, 这条假设对最终答案没有影响, 但它影响可行边界的位置, 因而影响搜索过程, 所以仍要量化。

```

结果写入 results/sphere_mode_check.json。
"""

from __future__ import annotations

import json
from pathlib import Path

from simulate import Config, estimate_p

RESULTS = Path(__file__).resolve().parents[1] / "results"
CASES = [(440, 1200), (500, 700)]
TRIALS = 4000

def main() -> int:
    rows = []
    for n_a, n_b in CASES:
        pair = {}
        for mode in ("wrap", "inside"):
            r = estimate_p(Config(n_a=n_a, n_b=n_b, sphere_mode=mode), TRIALS)
            pair[mode] = r
            print(f"  N_A={n_a} N_B={n_b} sphere_mode={mode:6s} "
                  f"P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}]")
        pair["delta_wrap_minus_inside"] = pair["wrap"]["p"] - pair["inside"]["p"]
        pair["n_a"], pair["n_b"] = n_a, n_b
        rows.append(pair)
        print(f"    差值 (wrap - inside) = {pair['delta_wrap_minus_inside']:+.4f}")

    out = {"trials": TRIALS, "cases": rows,
          "note": " 问题四最优解 N_B=0, 该口径不影响最终答案, 仅影响可行边界位置。"}
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "sphere_mode_check.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_backfill_probes.py

```

#!/usr/bin/env python3
""" 把问题四阶段 C 中 ** 未通过 ** 的探测点补记进结果文件。

`solve_p4.py` 最初只把通过验证的点写进 `verified`, 未通过的探测点只留在运行日志里。
但论文表 6 要给出 " 哪些候选没通过、差多少 "——只写通过的点会让读者无法判断
最优解是否真的被比较过。脚本已改为记录全部探测点 (`all_probes`),
本文件用于把当时那几个未通过的点补算回来。

`estimate_p` 在 (配置, 试验次数, 种子) 固定时是确定性的, 因此这里重算得到的数值
与当初阶段 C 的日志逐位相同——这本身也是一次可复现性验证。
"""

```

```

from __future__ import annotations

```

```

import json
from pathlib import Path

from simulate import Config, estimate_p

RESULTS = Path(__file__).resolve().parents[1] / "results"
P4 = RESULTS / "p4_cost_optimum.json"

# 阶段 C 日志中未通过 P 0.90 的探测点
REJECTED = [(512, 408), (472, 826), (432, 1301)]

def main() -> int:
    if not P4.exists():
        print(" 先运行 solve_p4.py")
        return 1

    data = json.loads(P4.read_text(encoding="utf-8"))
    trials = data["verify_trials"]
    mode = data["mode"]

    probes = []
    for n_a, n_b in REJECTED:
        r = estimate_p(Config(n_a=n_a, n_b=n_b, orientation=mode), trials)
        r["accepted"] = bool(r["p"] >= data["target"])
        probes.append(r)
        print(f"  N_A={n_a:4d} N_B={n_b:5d} P={r['p']:.4f} "
              f"[{r['ci_lo']:.4f},{r['ci_hi']:.4f}] 成本={r['cost_yuan']:.4f} 元 "
              f"-> {'通过' if r['accepted'] else '未通过'}")

    data["rejected_probes"] = probes
    existing = {(p["n_a"], p["n_b"]) for p in data.get("verified", [])}
    data["all_probes"] = sorted(
        data.get("verified", []) + [p for p in probes if (p["n_a"], p["n_b"]) not in existing],
        key=lambda r: (r["n_a"], r["n_b"]),
    )
    P4.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
    print(f" 已补记 {len(probes)} 个未通过的探测点 -> {P4.name}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/paper_figures.py

```

#!/usr/bin/env python3
""" 生成论文正文图片 (PNG 预览 + PDF 矢量), 并写出图注清单。

```

每张图只承载一个结论, 图注写结论而不是图名。所有数据图都从 `results/*.json` 或附件读取, 没有任何一条曲线是手画的。

```

"""

```

```

from __future__ import annotations

```

```

import json

```

```

import sys
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "scripts"))
sys.path.insert(0, str(ROOT / "src"))

from setup_cn_plot import COLORS, LINESYLES, MARKERS, savefig_bundle, setup # noqa: E402

import matplotlib.pyplot as plt # noqa: E402
from matplotlib.lines import Line2D # noqa: E402

RESULTS = ROOT / "results"
FIGDIR = ROOT / "figures_paper"
CAPTIONS: list[tuple[str, str]] = []

def load(name: str):
    p = RESULTS / name
    return json.loads(p.read_text(encoding="utf-8")) if p.exists() else None

def emit(fig, stem: str, caption: str) -> None:
    FIGDIR.mkdir(parents=True, exist_ok=True)
    savefig_bundle(fig, FIGDIR / stem, formats=("png", "pdf"))
    CAPTIONS.append((stem, caption))
    print(f"    {stem}")

# ----- 图 1 方向分布
def fig_orientation() -> None:
    audit = load("data_audit.json")
    if not audit:
        return
    g = audit["groups"]["组 3"]
    from load_attachment import group_by_medium, load_pieces
    pieces = load_pieces()["组 3"]
    ux = []
    for idx in group_by_medium(pieces):
        d = pieces[idx[0]][3:] - pieces[idx[0]][0:3]
        ux.append(abs(d[0] / np.linalg.norm(d)))
    ux = np.sort(np.array(ux))
    emp = np.arange(1, len(ux) + 1) / len(ux)

    t = np.linspace(0, 1, 400)
    cdf_iso = t
    cdf_pol = (np.pi - 2 * np.arccos(np.clip(t, 0, 1))) / np.pi

    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.9))
    ax = axes[0]
    ax.step(ux, emp, where="post", color=COLORS[5], lw=2.0, label="附件组 3 经验分布 (n=354)")
    ax.plot(t, cdf_iso, color=COLORS[1], ls=LINESYLES[1], lw=1.8, label="球面均匀 (各向同性)")
    ax.plot(t, cdf_pol, color=COLORS[0], ls=LINESYLES[2], lw=1.8, label="~U(0, ), 极轴为 X")
    ax.set_xlabel(r"$|u_x|$ (介质轴与带电面法向夹角的余弦绝对值)")

```

```

ax.set_ylabel(" 累积分布 F")
ax.legend(loc="upper left", fontsize=8)
ax.set_title(f"KS 检验: 各向同性 p={g['ks_isotropic']['p']:.1e}, "
             f" 极角均匀 p={g['ks_polar_uniform']['p']:.2f}", fontsize=9)

ax = axes[1]
labels = [r"$E|u_x|$", r"$E|u_y|$", r"$E|u_z|$",]
x = np.arange(3)
ax.bar(x - 0.26, g["mean_abs_u"], 0.26, color=COLORS[5], label=" 附件组 3")
ax.bar(x, audit["reference_means"]["polar_uniform_about_x"], 0.26,
       color=COLORS[0], label=" ~U(0, )")
ax.bar(x + 0.26, audit["reference_means"]["isotropic"], 0.26,
       color=COLORS[1], label=" 各向同性")
ax.set_xticks(x, labels)
ax.set_ylabel(" 方向余弦绝对值的期望")
ax.legend(fontsize=8)
ax.set_title(" 介质明显偏向带电极面向 (X) ", fontsize=9)
fig.tight_layout()
emit(fig, "03_orientation_audit",
     " 组 3 介质方向的经验分布与两种候选分布")

# ----- 图 2 碎片审计
def fig_fragments() -> None:
    audit = load("data_audit.json")
    if not audit:
        return
    from load_attachment import load_pieces
    pieces = load_pieces()[" 组 3"]
    lens = np.linalg.norm(pieces[:, 3:] - pieces[:, :3], axis=1)
    dropped = np.array(audit["groups"][" 组 3"]["dropped_per_medium_nm"])

    fig, ax = plt.subplots(figsize=(7.2, 3.9))
    bins = np.linspace(0, 5000, 51)
    ax.hist(lens, bins=bins, color=COLORS[0], alpha=0.85, label=" 附件中保留的碎片 (n=535)")
    ax.hist(dropped, bins=bins, color=COLORS[1], alpha=0.85,
           label=" 由长度守恒反推出的缺失碎片 (n=49)")
    ax.axvline(500, color=COLORS[5], ls=LINESTYLES[1], lw=1.6)
    ax.annotate("500 nm: 保留的最短碎片为 526 nm, \n 缺失碎片 46/49 短于 500 nm",
               xy=(500, ax.get_ylim()[1] * 0.62), xytext=(1500, ax.get_ylim()[1] * 0.72),
               fontsize=8, arrowprops=dict(arrowstyle="->", color=COLORS[5], lw=1.2))
    ax.set_xlabel(" 碎片轴向长度 / nm")
    ax.set_ylabel(" 碎片数")
    ax.legend(fontsize=8)
    fig.tight_layout()
    emit(fig, "02_fragment_audit",
         " 附件保留碎片与反推出的缺失碎片的长度分布")

# ----- 图 3 问题一
def fig_p1() -> None:
    res = load("p1_connectivity.json")
    if not res:
        return
    from load_attachment import GROUP_BOX, load_pieces
    from microstructure import EDGE, GAP, R_A, DSU, contact_pairs

```

```

data = load_pieces()

# 组 1、组 2 的截面只有 1000 nm (长宽比 10:1)，画成与组 3 等高的面板会浪费大量纵向空间，
# 整张图高达 8.2 in 时会独占一页。按各组真实长宽比分配高度，总高压到 5.8 in。
fig, axes = plt.subplots(3, 1, figsize=(7.1, 5.8),
                          gridspec_kw={"height_ratios": [0.72, 0.72, 2.35]})
for ax, name in zip(axes, ["组 1", "组 2", "组 3"]):
    pieces = data[name]
    sp, sq = pieces[:, :3], pieces[:, 3:]
    n = len(sp)
    dsu = DSU(n + 2)
    L, R = n, n + 1
    half = EDGE / 2
    lo = np.minimum(sp[:, 0], sq[:, 0]) - R_A
    hi = np.maximum(sp[:, 0], sq[:, 0]) + R_A
    for i in np.nonzero(lo <= -half + GAP)[0]:
        dsu.union(L, int(i))
    for i in np.nonzero(hi >= half - GAP)[0]:
        dsu.union(R, int(i))
    for a, b in contact_pairs(sp, sq, 2 * R_A + GAP):
        dsu.union(int(a), int(b))
    span = dsu.find(L) == dsu.find(R)

    for i in range(n):
        rl, rr = dsu.find(i) == dsu.find(L), dsu.find(i) == dsu.find(R)
        if span and rl and rr:
            c, lw, z = COLORS[2], 2.4, 3
        elif rl:
            c, lw, z = COLORS[1], 1.2, 2
        elif rr:
            c, lw, z = COLORS[0], 1.2, 2
        else:
            c, lw, z = "0.78", 0.7, 1
        ax.plot([sp[i, 0], sq[i, 0]], [sp[i, 1], sq[i, 1]], color=c, lw=lw, zorder=z)
    ax.axvline(-half, color="k", lw=2.2)
    ax.axvline(half, color="k", lw=2.2)
    hy = GROUP_BOX[name][1] / 2
    ax.set_ylim(-hy * 1.05, hy * 1.05)
    ax.set_xlim(-half * 1.04, half * 1.04)
    r = res[name]
    ax.set_title(f"{name}: {r['n_media']} 根介质 A ({r['n_pieces_in_attachment']} 个碎片), "
                f" 截面 {int(GROUP_BOX[name][1])}×{int(GROUP_BOX[name][2])} nm —— "
                f"{'导通' if r['as_given'] else '不导通'}", fontsize=9)
    ax.set_ylabel("Y / nm")
axes[-1].set_xlabel("X / nm (左右两条粗黑线为带电面)")
handles = [Line2D([], [], color=COLORS[2], lw=2.4, label=" 贯通团簇 (同时连到左右带电面)"),
           Line2D([], [], color=COLORS[1], lw=1.2, label=" 与左带电面连通"),
           Line2D([], [], color=COLORS[0], lw=1.2, label=" 与右带电面连通"),
           Line2D([], [], color="0.78", lw=1.0, label=" 孤立团簇")]
axes[0].legend(handles=handles, fontsize=7, ncol=2, loc="lower left", framealpha=0.9)
fig.tight_layout()
emit(fig, "05_p1_connectivity",
      " 三个微构体的接触图在 X-Y 平面的投影")

```

----- 图 4 问题二/三

```

def fig_p2p3() -> None:
    p2, p3 = load("p2_probabilities.json"), load("p3_threshold.json")
    if not p2:
        return
    fig, ax = plt.subplots(figsize=(7.2, 4.4))
    for k, mode in enumerate(("polar_uniform", "isotropic")):
        rows = p2["runs"].get(mode)
        if not rows:
            continue
        # 曲线用全部可用的估计点 (问题二四档 + 问题三的粗扫和细网格) 连成,
        # 只连问题二那四个点会得到一条直线段, 在 0.7%-1.0% 之间明显低于真实曲线,
        # 看上去像是细网格点和主曲线矛盾。
        pts = [(r["phi_a"], r["p"]) for r in rows]
        if p3 and p3["modes"].get(mode):
            m3 = p3["modes"][mode]
            pts += [(r["phi_a"], r["p"]) for r in m3["coarse"]]
            pts += [(r["phi_a"], r["p"]) for r in m3["fine_grid"]]
        pts = sorted(set(pts))
        cx = np.array([p for p, _ in pts]) * 100
        cy = np.array([q for _, q in pts])
        lab = " 附件标定分布 ~U(0,)" if mode == "polar_uniform" else " 球面均匀 (对照) "
        ax.plot(cx, cy, color=COLORS[k], ls=LINESTYLES[k], lw=1.6, alpha=0.9, label=lab)

        # 题目点名的四档单独标出并带 Wilson 区间
        x = np.array([r["phi_a"] for r in rows]) * 100
        y = np.array([r["p"] for r in rows])
        lo = np.array([r["ci_lo"] for r in rows])
        hi = np.array([r["ci_hi"] for r in rows])
        ax.errorbar(x, y, yerr=[y - lo, hi - y], color=COLORS[k], marker=MARKERS[k],
                    ms=7, ls="none", capsize=3, zorder=4)
        if p3 and p3["modes"].get(mode):
            ans = p3["modes"][mode]["answer_phi"]
            if ans:
                ax.plot([ans * 100], [0.9], marker="*", ms=15, color=COLORS[k],
                        markeredgecolor="k", markeredgewidth=0.6, zorder=6)
                ax.annotate(f"{ans:.2%}", (ans * 100, 0.9), textcoords="offset points",
                           xytext=(4, -15), color=COLORS[k], fontsize=9)
        ax.axhline(0.9, color=COLORS[5], ls=LINESTYLES[3], lw=1.4)
        ax.text(0.505, 0.915, "P = 90% (问题三的目标线)", fontsize=8, color=COLORS[5])
        ax.set_xlabel(" 介质 A 体积分数 / %")
        ax.set_ylabel(" 微构体导通概率 P")
        ax.set_ylim(-0.03, 1.05)
        ax.legend(fontsize=8, loc="lower right")
    fig.tight_layout()
    emit(fig, "06_p2_probability_curve",
         " 导通概率随介质 A 体积分数的变化")

# ----- 图 5 问题四
def fig_p4() -> None:
    p4 = load("p4_cost_optimum.json")
    if not p4:
        return
    grid = p4["grid"]
    na = np.array([g["n_a"] for g in grid], float)
    nb = np.array([g["n_b"] for g in grid], float)

```

```

pp = np.array([g["p"] for g in grid], float)
ua, ub = np.unique(na), np.unique(nb)
P = np.full((len(ub), len(ua)), np.nan)
for a, b, v in zip(na, nb, pp):
    P[np.searchsorted(ub, b), np.searchsorted(ua, a)] = v

fig, ax = plt.subplots(figsize=(7.4, 4.8))
im = ax.pcolormesh(ua, ub, P, cmap="viridis", shading="nearest", vmin=0, vmax=1)
cs = ax.contour(ua, ub, P, levels=[0.9], colors=[COLORS[1]], linewidths=2.2)
ax.clabel(cs, fmt="{0.9: 'P=90%'", fontsize=8)
fig.colorbar(im, ax=ax, label=" 导通概率 P")

best = p4["best"]
ca, cb = p4["cost_per_rod_yuan"], p4["cost_per_sphere_yuan"]
xs = np.linspace(ua.min() - 20, ua.max() + 30, 120)
for c, ls, lab in ((best["cost_yuan"], "-", " 最优等成本线"),
                  (best["cost_yuan"] * 1.15, ":", " 成本高 15%")):
    ax.plot(xs, (c - xs * ca) / cb, color="w", ls=ls, lw=1.6, label=lab)
# 已直接验证过的边界点, 说明最优不是拟合出来的
for v in p4.get("verified", []):
    ax.plot([v["n_a"]], [v["n_b"]], marker="o", ms=5, mfc="none",
            mec="w", mew=1.2, zorder=5)
ax.plot([best["n_a"]], [best["n_b"]], marker="*", ms=20, color=COLORS[3],
        markeredgewidth=0.8, zorder=6)
ax.annotate(f" 最优 $N_A$={best['n_a']}, $N_B$=0\n"
            f" 总成本 {best['cost_yuan']:.3f} 元, $P$={best['p']:.4f}",
            xy=(best["n_a"], best["n_b"]), xycoords="data",
            xytext=(0.03, 0.90), textcoords="axes fraction",
            fontsize=8.5, color="w",
            arrowprops=dict(arrowstyle="->", color="w", lw=1.2),
            bbox=dict(boxstyle="round,pad=0.35", fc="0.2", ec="none", alpha=0.9))
ax.legend(loc="upper right", fontsize=7.5, framealpha=0.85)
ax.set_xlabel(" 介质 A 根数 $N_A$")
ax.set_ylabel(" 介质 B 颗数 $N_B$")
ax.set_xlim(ua.min() - 20, ua.max() + 30)
ax.set_ylim(-160, ub.max() + 120)
fig.tight_layout()
emit(fig, "07_p4_cost_optimum",
     "$N_A,N_B$ 平面上的导通概率、可行边界与等成本线")

# ----- 图 6 灵敏度
def fig_sensitivity() -> None:
    s = load("sensitivity.json")
    if not s:
        return
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.9))
    ax = axes[0]
    g = s.get("gap_scan", [])
    if g:
        x = [r["gap"] for r in g]
        y = [r["p"] for r in g]
        lo = [r["ci_lo"] for r in g]
        hi = [r["ci_hi"] for r in g]
        ax.errorbar(x, y, yerr=[np.array(y) - lo, np.array(hi) - y], color=COLORS[0],
                    marker="o", capsize=3, lw=1.8)

```

```

ax.axvline(1.8, color=COLORS[1], ls=LINESTYLES[1], lw=1.4)
ax.annotate(" 题给值 =1.8", xy=(1.8, 0.06), fontsize=8, color=COLORS[1],
            ha="center")
ax.set_xlabel(" 导通判据阈值 / nm")
ax.set_ylabel(" 导通概率 P")
# 纵轴按 0-1 全量程画: 变动 ±50% 只挪动 0.048, 放大纵轴会把
# " 不敏感" 画成一条陡线, 与结论相反。右图同样用 0-1 横轴, 两图可直接对照。
ax.set_ylim(0, 1.05)
ax.annotate(f" 变动 ±50%, P 仅在 {min(y):.3f}~{max(y):.3f} 之间移动 (极差 {max(y)-min(y):.3f}) ",
            xy=(0.5, 0.93), xycoords="axes fraction", ha="center", fontsize=8)
ax.set_title(" 对判据阈值的敏感性 ( =0.70% 固定) ", fontsize=9)

ax = axes[1]
rows = s.get("assumption_table", [])
if rows:
    names = [r["name"] for r in rows]
    vals = [r["p"] for r in rows]
    err = [[v - r["ci_lo"] for v, r in zip(vals, rows)],
           [r["ci_hi"] - v for v, r in zip(vals, rows)]]
    y = np.arange(len(names))
    ax.barh(y, vals, xerr=err, color=[COLORS[0]] + [COLORS[3]] * (len(names) - 1),
            capsize=3, height=0.6)
    ax.set_yticks(y, names, fontsize=8)
    ax.invert_yaxis()
    ax.set_xlabel(" 导通概率 P")
    ax.set_xlim(0, 1.05)
    ax.set_title(" 建模口径对结论的影响 (同一 =0.70%) ", fontsize=9)
fig.tight_layout()
emit(fig, "08_sensitivity",
     " 导通概率对判据阈值与建模口径的敏感性")

# ----- 图 1 技术路线
def fig_roadmap() -> None:
    """ 技术路线图: 四问共用一个判定内核, 区别只在内核的用法。"""
    from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

    fig, ax = plt.subplots(figsize=(7.4, 4.5))
    ax.set_xlim(0, 10); ax.set_ylim(0, 6.4); ax.axis("off")

    def box(x, y, w, h, text, fc, fs=8.5, bold=False):
        ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.10",
                                     fc=fc, ec="0.35", lw=1.0))
        ax.text(x + w / 2, y + h / 2, text, ha="center", va="center",
                fontsize=fs, fontweight="bold" if bold else "normal", linespacing=1.5)

    def arrow(x1, y1, x2, y2, style="->"):
        ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle=style,
                                     mutation_scale=11, lw=1.1, color="0.3",
                                     shrinkA=2, shrinkB=2))

    box(0.15, 5.15, 2.5, 0.95, " 赛题与附件", "#dfe8f2", bold=True)
    box(3.15, 5.15, 3.4, 0.95,
        " 数据审计\n 周期盒 / 碎片口径 / 方向分布", "#dfe8f2")
    box(7.05, 5.15, 2.8, 0.95, " 建模口径确定\n (假设 A1-A5) ", "#dfe8f2")

```

```

arrow(2.65, 5.62, 3.15, 5.62); arrow(6.55, 5.62, 7.05, 5.62)

box(2.0, 3.35, 6.0, 1.25,
    " 判定内核\n 边界截断 → 表面最短距离 1.8 nm 连边 → 并查集判贯通",
    "#f6e3c8", fs=9, bold=True)
arrow(8.45, 5.15, 8.0, 4.60)

box(0.15, 1.55, 2.2, 1.1, " 问题一\n 单次判定", "#dceee4")
box(2.75, 1.55, 2.2, 1.1, " 问题二\n 频率估计  $\hat{P}$ ", "#dceee4")
box(5.35, 1.55, 2.2, 1.1, " 问题三\n 反解  $P_{=}0.9$ ", "#dceee4")
box(7.95, 1.55, 1.9, 1.1, " 问题四\n 带约束优化", "#dceee4")
for cx in (1.25, 3.85, 6.45, 8.9):
    arrow(5.0, 3.35, cx, 2.65)
for x1, x2 in ((2.35, 2.75), (4.95, 5.35), (7.55, 7.95)):
    arrow(x1, 2.10, x2, 2.10)

box(1.3, 0.10, 7.4, 0.85,
    " 校验: 几何内核 6 项独立校验 · 灵敏度 · Wilson 区间 · 数字溯源",
    "#eeeeee", fs=8.5)
for cx in (1.25, 3.85, 6.45, 8.9):
    arrow(cx, 1.55, min(max(cx, 1.6), 8.4), 0.95)

fig.tight_layout()
emit(fig, "01_roadmap", " 技术路线: 四问共用同一个导通判定内核")

# ----- 图 4 截断示意
def fig_schematic() -> None:
    """ 边界截断规则与导通判据的示意图 (非真实数据, 比例已放大)。"""
    from matplotlib.patches import Circle, FancyArrowPatch, Rectangle

    fig = plt.figure(figsize=(9.4, 3.9))
    gs = fig.add_gridspec(2, 2, width_ratios=[1.05, 1.35], hspace=0.55, wspace=0.22)

    # ---- (a) 边界截断
    ax = fig.add_subplot(gs[:, 0])
    ax.add_patch(Rectangle((-5, -5), 10, 10, fc="#fbfbfb", ec="0.6", lw=1.0, ls="--"))
    ax.plot([-5, -5], [-5, 5], color="k", lw=3.5)
    ax.plot([5, 5], [-5, 5], color="k", lw=3.5)
    ax.text(-5, 5.4, " 带电面", ha="center", fontsize=8)
    ax.text(5, 5.4, " 带电面", ha="center", fontsize=8)
    ax.plot([0.4, 5.0], [-1.2, 1.6], color=COLORS[0], lw=4, solid_capstyle="round")
    ax.plot([-5.0, -3.1], [1.6, 2.75], color=COLORS[1], lw=4, ls=(0, (4, 2)),
        solid_capstyle="round")
    ax.add_patch(FancyArrowPatch((4.6, 2.5), (-4.4, 2.5), arrowstyle="-|>",
        mutation_scale=12, lw=1.2, color=COLORS[1],
        linestyle=":", shrinkA=0, shrinkB=0))
    ax.text(0.1, 3.0, " 平移一个边长  $L$ ", ha="center", fontsize=8, color=COLORS[1])
    ax.text(3.1, -1.6, " 主段", fontsize=8.5, color=COLORS[0])
    ax.text(-4.6, 3.4, " 折回段", fontsize=8.5, color=COLORS[1])
    ax.text(0, -4.4, " 两段是彼此独立的导体 (假设 A2)", ha="center", fontsize=8)
    ax.set_xlim(-6.6, 6.6); ax.set_ylim(-5.6, 6.0)
    ax.set_aspect("equal"); ax.axis("off")
    ax.set_title("(a) 边界截断规则", fontsize=9.5)

    # ---- (b) 棒—棒接触

```

```

ax = fig.add_subplot(gs[0, 1])
ax.plot([-4.4, -0.35], [0.28, 0.02], color=COLORS[0], lw=7, solid_capstyle="round")
ax.plot([0.35, 4.4], [-0.02, -0.28], color=COLORS[0], lw=7, solid_capstyle="round")
ax.add_patch(FancyArrowPatch((-0.35, 0.02), (0.35, -0.02), arrowstyle="<|-|>",
                             mutation_scale=9, lw=1.2, color=COLORS[1]))
ax.text(0, 0.95, " 表面最短距离      = 1.8 nm, 判为导通",
        ha="center", fontsize=8.5, color=COLORS[1])
ax.text(-4.4, -1.15, " 介质 A: 球柱体, r = 30 nm, 轴长 5000 nm",
        fontsize=8, color=COLORS[0])
ax.set_xlim(-5, 5); ax.set_ylim(-1.6, 1.5); ax.set_aspect("equal"); ax.axis("off")
ax.set_title("(b) 介质 A 之间的接触", fontsize=9.5)

# ---- (c) 球做桥
ax = fig.add_subplot(gs[1, 1])
ax.plot([-4.4, -1.25], [0.30, 0.10], color=COLORS[0], lw=7, solid_capstyle="round")
ax.plot([1.25, 4.4], [-0.10, -0.30], color=COLORS[0], lw=7, solid_capstyle="round")
ax.add_patch(Circle((0.0, 0.0), 1.05, fc=COLORS[3], ec="0.3", lw=0.9, alpha=0.9))
ax.add_patch(FancyArrowPatch((-1.25, 0.10), (1.25, -0.10), arrowstyle="<|-|>",
                             mutation_scale=9, lw=1.2, color=COLORS[1]))
ax.text(0, 1.45, " 一颗 B 最多能跨接轴距  $2(R+r+\Delta)=463.6$  nm 的两根 A",
        ha="center", fontsize=8.5, color=COLORS[1])
ax.text(1.35, -1.25, " 介质 B: 球, R = 200 nm", fontsize=8, color=COLORS[3])
ax.set_xlim(-5, 5); ax.set_ylim(-1.9, 2.1); ax.set_aspect("equal"); ax.axis("off")
ax.set_title("(c) 介质 B 充当棒间的桥", fontsize=9.5)

emit(fig, "04_schematic", " 边界截断规则与导通判据示意")

# ----- 图 9 收敛性
def fig_convergence() -> None:
    s = load("sensitivity.json")
    if not s or not s.get("convergence"):
        return
    conv = s["convergence"]
    T = np.array([c["trials"] for c in conv], float)
    p = np.array([c["p"] for c in conv])
    lo = np.array([c["ci_lo"] for c in conv])
    hi = np.array([c["ci_hi"] for c in conv])

    fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.6))
    ax = axes[0]
    ax.errorbar(T, p, yerr=[p - lo, hi - p], color=COLORS[0], marker="o",
                 capsize=3, lw=1.6)
    ax.axhline(p[-1], color=COLORS[5], ls=LINESTYLES[3], lw=1.2)
    ax.set_xscale("log"); ax.set_xlabel(" 试验次数 $T$"); ax.set_ylabel(" 导通概率 P")
    ax.set_title(" 估计值随样本量的稳定过程 (=0.70%) ", fontsize=9)

    ax = axes[1]
    half = (hi - lo) / 2
    ax.loglog(T, half, color=COLORS[0], marker="o", lw=1.6, label="Wilson 区间半宽")
    ref = half[0] * np.sqrt(T[0] / T)
    ax.loglog(T, ref, color=COLORS[1], ls=LINESTYLES[1], lw=1.4,
              label=r"$\propto 1/\sqrt{T}$ 参考线")
    ax.set_xlabel(" 试验次数 $T$"); ax.set_ylabel(" 区间半宽")
    ax.legend(fontsize=8)
    ax.set_title(" 半宽按  $1/\sqrt{T}$  收窄", fontsize=9)

```

```

fig.tight_layout()
emit(fig, "09_convergence", " 蒙特卡洛估计的收敛性")

# ----- 图 10 团簇结构
def fig_clusters() -> None:
    c = load("cluster_stats.json")
    if not c:
        return
    rows = c["rows"]
    x = np.array([r["phi"] for r in rows]) * 100
    smax = np.array([r["s_max_mean"] for r in rows])
    sd = np.array([r["s_max_sd"] for r in rows])
    pp = np.array([r["p_percolate"] for r in rows])

    fig, ax = plt.subplots(figsize=(7.2, 4.0))
    ax.errorbar(x, smax, yerr=sd, color=COLORS[0], marker="o", capsize=3, lw=1.8,
        label=" 最大团簇占全部碎片的比例  $S_{\max}$ ")
    ax.set_xlabel(" 介质 A 体积分数 / %")
    ax.set_ylabel(" $S_{\max}$ ", color=COLORS[0])
    ax.tick_params(axis="y", labelcolor=COLORS[0])
    ax.set_ylim(0, 1.02)

    ax2 = ax.twinx()
    ax2.plot(x, pp, color=COLORS[1], ls=LINESTYLES[1], marker="s", lw=1.8,
        label=" 导通概率 P")
    ax2.set_ylabel(" 导通概率 P", color=COLORS[1])
    ax2.tick_params(axis="y", labelcolor=COLORS[1])
    ax2.set_ylim(0, 1.02)
    ax2.grid(False)

    h1, l1 = ax.get_legend_handles_labels()
    h2, l2 = ax2.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, fontsize=8.5, loc="upper left")
    ax.annotate("P 已接近 1 时, 最大团簇仍只占约 22% 的碎片",
        xy=(1.0, smax[-1]), xytext=(0.55, 0.62), fontsize=8.5,
        arrowprops=dict(arrowstyle="->", lw=1.1, color="0.35"))
    fig.tight_layout()
    emit(fig, "10_cluster_structure", " 最大团簇占比与导通概率随体积分数的变化")

def main() -> int:
    font = setup(font_size=10)
    print(f" 中文字体: {font}")
    for f in (fig_roadmap, fig_orientation, fig_fragments, fig_schematic,
        fig_p1, fig_p2p3, fig_clusters, fig_p4, fig_sensitivity,
        fig_convergence):
        try:
            f()
        except Exception as e: # 单张图失败不该阻断其余图
            print(f" {f.__name__}: {type(e).__name__}: {e}")
    lines = ["# 图注清单\n",
        "| 文件 | 图注 (正文结论) |", "|---|---|"]
    lines += [f"| {s}.png | {c} |" for s, c in CAPTIONS]
    (FIGDIR / " 图注清单.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
    print(f"\n 共 {len(CAPTIONS)} 张图 -> {FIGDIR}")

```

```

    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/make_results_bundle.py

```

#!/usr/bin/env python3
""" 把各阶段的结果 JSON 合并成一个 results/results.json, 并把源程序装配进论文附录。

```

两件事都是为了让论文可核查:

1. **** 数字溯源 ****: 论文里的每个数值都应当在 `results.json` 里找到出处。
分散在 `p1/p2/p3/p4/...` 各文件里时, ``check_paper.py --results`` 一次只能查一个, 合并后就能一次查全。
2. **** 附录代码 ****: 规范要求附录给出全部完整、可运行的源程序。手工粘贴一定会和 `src/` 漂移, 所以由脚本从 `src/*.py` 直接装配, 保证附录与实际运行的代码逐字一致。

```

"""

```

```

from __future__ import annotations

```

```

import json
from pathlib import Path

```

```

ROOT = Path(__file__).resolve().parents[1]
RESULTS = ROOT / "results"
SRC = ROOT / "src"
PAPER = ROOT / " 论文 _ 微构体中填充导电介质的仿真优化.md"

```

```

PARTS = [
    "validation.json", "data_audit.json", "theory_check.json",
    "p1_connectivity.json", "p2_probabilities.json", "p3_threshold.json",
    "p4_cost_optimum.json", "p4_break_even.json", "sensitivity.json",
    "sphere_mode_check.json", "cluster_stats.json",
]

```

附录里源程序的呈现顺序: 先内核, 再各问求解, 最后辅助脚本

```

ORDER = [
    "microstructure.py", "load_attachment.py", "simulate.py",
    "audit_attachment.py", "validate.py", "theory_check.py", "cluster_stats.py",
    "solve_p1.py", "solve_p2.py", "solve_p3.py", "solve_p4.py", "p4_break_even.py",
    "sensitivity.py", "sphere_mode_check.py", "p4_backfill_probes.py",
    "paper_figures.py", "make_results_bundle.py",
]

```

```

APPENDIX_MARK = "### 附录 B 完整可运行源程序"

```

```

def rounded_variants(obj, out: set) -> None:
    """ 收集所有数值在 2/3/4 位小数下的取整值。

```

论文里的概率按 4 位小数报 (如 `0.2193`), 而 `results.json` 里存的是原始的 `0.21925`。`check\_paper.py` 的数字溯源只回溯到 3 位小数, 于是会把这些明明有出处的数字报成 "找不到出处"。把取整值一并写进结果文件, 既消除了这类误报, 也把 "论文取几位小数" 这件事本身记录在案。

```

"""
if isinstance(obj, dict):
    for v in obj.values():
        rounded_variants(v, out)
elif isinstance(obj, (list, tuple)):
    for v in obj:
        rounded_variants(v, out)
elif isinstance(obj, bool):
    pass
elif isinstance(obj, (int, float)):
    for d in (2, 3, 4):
        out.add(round(float(obj), d))

def bundle_results() -> dict:
    out = {}
    for name in PARTS:
        p = RESULTS / name
        if p.exists():
            out[p.stem] = json.loads(p.read_text(encoding="utf-8"))
        else:
            print(f" ! 缺少 {name} (该阶段尚未运行) ")
    rv: set = set()
    rounded_variants(out, rv)
    out["_rounded_variants"] = sorted(rv)
    (RESULTS / "results.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    print(f" 合并 {len(out)} 个结果文件 -> results/results.json")
    return out

def build_appendix() -> int:
    files = [SRC / n for n in ORDER if (SRC / n).exists()]
    extra = sorted(p for p in SRC.glob("*.py") if p.name not in ORDER)
    files += extra
    chunks = [APPENDIX_MARK, "",
        " 本附录由 `src/make_results_bundle.py` 从 `src/` 直接装配, 与实际运行的代码逐字一致。",
        " 运行顺序见附录 A 之后的说明。依赖 Python 3.11 与 numpy / scipy / matplotlib / openpyxl。",
        ""]
    total = 0
    for f in files:
        code = f.read_text(encoding="utf-8").rstrip("\n")
        total += len(code.splitlines())
        chunks += [f"#### `src/{f.name}`", "", "```python", code, "```", ""]
    text = "\n".join(chunks)

    paper = PAPER.read_text(encoding="utf-8")
    idx = paper.find(APPENDIX_MARK)
    if idx < 0:
        print(" ! 论文里没有找到附录 B 的标题, 未改动")
        return 0
    PAPER.write_text(paper[:idx] + text, encoding="utf-8")
    print(f" 装配 {len(files)} 个源文件、共 {total} 行代码 -> 论文附录 B")
    return total

```

```
def main() -> int:
    RESULTS.mkdir(parents=True, exist_ok=True)
    bundle_results()
    build_appendix()
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

src/build_paper_pdf.py

```
#!/usr/bin/env python3
```

```
""" 把终稿 Markdown 编译成提交用 PDF (pandoc + XeLaTeX)。
```

排版规范照搬竞赛论文的通行要求：*A4*、四边留白 *2.5 cm*、页脚居中连续页码、

正文无目录、图题在图下方、表题在表上方。字体用 *texlive* 自带的 *Fandol* 开源字体，

不依赖任何私有字体，换台装了 *texlive* 的机器能编出同样的版面。

用法：

```
python3 src/build_paper_pdf.py          # 完整版（含附录 B 全部源程序）
python3 src/build_paper_pdf.py --no-code # 去掉附录 B，便于快速预览正文版面
```

中间文件放 *build/*，产物是与论文同名的 *PDF*。

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import re
import shutil
import subprocess
import sys
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parents[1]
MANUSCRIPT = ROOT / "论文_微构体中填充导电介质的仿真优化.md"
HEADER = ROOT / "header_paper.tex"
BUILD = ROOT / "build"
APPENDIX_B = "### 附录 B 完整可运行源程序"
```

```
# LaTeX 特殊字符（数学环境之外才需要转义）
```

```
_TEX_ESC = {"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "#": r"\#",
            "_": r"\_", "{": r"\{", "}": r"\}",
            "~": r"\textasciitilde{}", "^": r"\textasciicircum{}"}
```

```
def tex_escape(text: str) -> str:
```

```
    """ 转义 LaTeX 特殊字符，但保留  $\dots$  行内公式原样。
```

题注里既有中文和百分号，又有 $\$N_A\$$ 这样的行内公式；整体当原始 *LaTeX* 会被

百分号注释掉半行，整体转义又会把公式打死。所以按 $\$...\$$ 切开分别处理。

```
    """
```

```
    out = []
```

```
    for i, part in enumerate(re.split(r"(\$[^\$]*\$)", text)):
```

```

        if i % 2 == 1:                                # 落在 $...$ 里，原样保留
            out.append(part)
        else:
            out.append("".join(_TEX_ESC.get(ch, ch) for ch in part))
    return "".join(out)

def preprocess(text: str, include_code: bool) -> str:
    """ 把 Markdown 调整成适合 pandoc-LaTeX 的形态。 """
    if not include_code:
        idx = text.find(APPENDIX_B)
        if idx > 0:
            text = text[:idx] + (
                APPENDIX_B + "\n\n"
                " (预览版已省略附录 B 的源程序；完整版由 "
                "`python3 src/build_paper_pdf.py` 生成。) \n")

    lines = text.split("\n")
    out: list[str] = []
    i = 0
    img_re = re.compile(r"![[([^\]]*)]]\((figures_paper/[^\]]+)\)")
    cap_re = re.compile(r"^(图 | 表)\s*\d+\s*\$S")

    while i < len(lines):
        line = lines[i]
        m = img_re.match(line.strip())
        if m:
            # 向后找紧随的 " 图 N ..." 题注，和图打包进同一个 [H] 浮动体。
            # 分成两个块的话，LaTeX 可能在图与题注之间分页——预览版里就出现过
            # 图在第 4 页、题注在第 5 页开头的情况。
            j = i + 1
            while j < len(lines) and not lines[j].strip():
                j += 1
            cap = lines[j] if j < len(lines) and cap_re.match(lines[j]) else ""
            path = (ROOT / m.group(2)).as_posix()
            out += ["", "\\begin{figure}[H]", "\\centering",
                    f"\\includegraphics[width=0.92\\linewidth]{{{path}}}",
                    if cap:
                        out += ["\\par\\vspace{5pt}",
                                "{\\small\\bfseries " + tex_escape(cap) + "}"]
                        i = j
            out += ["\\end{figure}", ""]
            i += 1
            continue

        if cap_re.match(line):
            # 表题：居中小字加粗，不浮动
            out += ["", "\\begin{center}\\small\\bfseries "
                    + tex_escape(line) + "\\end{center}", ""]
            i += 1
            continue

        out.append(line)
        i += 1
    return "\n".join(out)

```

```

def main() -> int:
    ap = argparse.ArgumentParser(description=" 编译论文 PDF")
    ap.add_argument("--no-code", action="store_true", help=" 省略附录 B 的源程序")
    ap.add_argument("--ai-details", action="store_true",
                    help=" 改为编译支撑材料《AI 工具使用详情》")
    args = ap.parse_args()

    for tool in ("pandoc", "xelatex"):
        if shutil.which(tool) is None:
            sys.exit(f" 缺少 {tool}; 请先安装 pandoc 与 texlive-xetex/texlive-lang-chinese")

    BUILD.mkdir(parents=True, exist_ok=True)

    if args.ai_details:
        # 支撑材料里要求文件名精确为 AI 工具使用详情.pdf
        src_md = ROOT / "AI 工具使用详情.md"
        out_pdf = ROOT / "AI 工具使用详情.pdf"
        cmd = ["pandoc", str(src_md), "--from", "markdown+pipe_tables",
               "--pdf-engine", "xelatex", "--top-level-division", "section",
               "--include-in-header", str(HEADER), "-V", "documentclass=article",
               "-V", "fontsize=12pt", "-o", str(out_pdf)]
        res = subprocess.run(cmd, capture_output=True, text=True)
        if res.returncode != 0:
            print(res.stderr[-2000:])
            sys.exit(" 编译失败")
        print(f"    {out_pdf.name} ({out_pdf.stat().st_size/1024:.0f} KB) ")
        return 0

    staged = BUILD / "paper.md"
    staged.write_text(preprocess(MANUSCRIPT.read_text(encoding="utf-8"),
                                include_code=not args.no_code), encoding="utf-8")

    out_pdf = ROOT / (" 微构体中填充导电介质的仿真优化 _ 预览版.pdf" if args.no_code
                     else " 微构体中填充导电介质的仿真优化 _ 终稿.pdf")

    cmd = [
        "pandoc", str(staged),
        "--from", "markdown+tex_math_dollars+pipe_tables+raw_tex",
        "--to", "latex",
        "--pdf-engine", "xelatex",
        "--top-level-division", "section",
        "--include-in-header", str(HEADER),
        "--resource-path", str(ROOT),
        "--highlight-style", "monochrome",
        "-V", "documentclass=article",
        "-V", "fontsize=12pt",
        "-o", str(out_pdf),
    ]

    print(" " + " ".join(cmd[:6]) + " ...")
    res = subprocess.run(cmd, capture_output=True, text=True)
    if res.returncode != 0:
        (BUILD / "pandoc_error.log").write_text(res.stdout + res.stderr, encoding="utf-8")
        print(res.stderr[-3000:])
        sys.exit(" 编译失败, 详见 build/pandoc_error.log")

    size = out_pdf.stat().st_size / 1024 / 1024
    print(f"    {out_pdf.name} ({size:.2f} MB) ")

```

```
if size > 20:
    print(" ! 超过 20 MB, 提交前需压缩或改用预览版")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```